Advanced Web Development

2- REACT Basics

Jeannette Chin

Learning Objectives

- Evaluate REACT rendering
- Write REACT component in JSX
- Evaluate JSX syntax and rules
- Write conditional loops in JSX
- Evaluate REACT props
- Event handling
- Component Memory in REACT
- Implement useState hook



The evolution of REACT: v16 to >v19

- Frontend JavaScript framework for building user interfaces
- React's architectural journey has shifted from manual DOM manipulation to concurrent rendering and server-first architectures.
- React v16-17 (legacy base) – hooks useState, useEffect
- React v18 (concurrent era) - automatic batching
- Modern Standard is >v.19
  - Features React Server Components (RSC)
  - built-in Actions for forms,
  - the use() API,
  - automatic memorisation via the React Compiler,
  - direct ref prop passing without forwardRef.



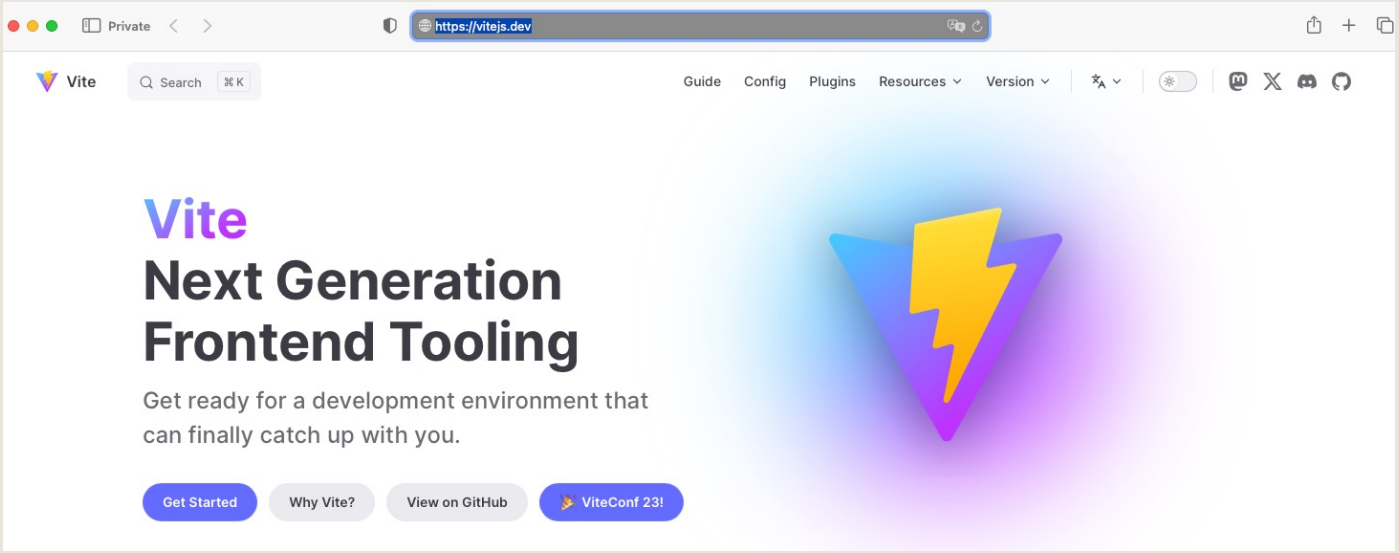
REACT >v19

- The concept of ‘components’ --- > in practice, it referred to JavaScript functions.
  - Learning REACT is learning JavaScript programming through TypeScript
- The concept of ‘JSX’ (JavaScript XML)– markup code in REACT, including HTML
- Platform specific – Web (REACT) or Mobile (REACT Native)

Vite for scaffolding

```
npm create vite@latest first-project --
--template react-ts
```

- Developed by the creator of Vue
- Use native ES modules to bundle JS files and application assets, dramatically speed up the development time.
- Use Vite - https://vitejs.dev



Prerequisites

- HTML, CSS, TypeScript
- Won’t cover HTML and CSS
- JavaScript - https://developer.mozilla.org/en-US/docs/Learn/JavaScript

‘Unlearning everything’

- In the last two decades we learned that the separation of concerns is important
  - Separation the logic from template
  - Inline CSS and Embedded JS are discouraged
  - Write JS , CSS and HTML in separate files.

- In the world of REACT
  - Putting everything where it belongs – next to the logic
  - Organise the application by composing small bricks called ‘components’
  - Classic concept of separation of concerns has taken over by the modern ‘component-based’ approach

REACT ‘render’ method

```
render() {
  return (
    <button style={{ color: 'red' }} onClick={this.handleClick}>
      Click me!
    </button>
  )
}
```

- Mixing JS, HTML and CSS !

REACT Component

REACT since v18

“Component is the base class for the React components defined as JavaScript classes. Class components are still supported by React, but we don’t recommend using them in new code.”

- Pre-REACT v18 applications are advised to migrate : https://react.dev/reference/react/Component#alternatives

- Component === function

REACT Component - legacy

```tsx
// ❌ Legacy Class Component Pattern (Do not use in modern codebases)
import React, { Component } from 'react';

interface CounterProps { initialCount: number; }
interface CounterState { count: number; }

export class LegacyCounter extends Component<CounterProps, CounterState> {
  state: CounterState = { count: this.props.initialCount };

  increment = () => {
    this.setState((prev) => ({ count: prev.count + 1 }));
  };

  render() {
    return <button onClick={this.increment}>Count: {this.state.count}</button>;
  }
}
```

REACT Component - pure JS

src > UserProfileCard.jsx > ...

```jsx
import React from "react";

// ❌ Issues: No build-time errors if 'user' missing fields or 'onUpdateRole'
// is called with invalid parameters.
export function UserProfileCard({ user, onUpdateRole }) {
  return (
    <div className="profile-card">
      <h2>{user.name.toUpperCase()}</h2>
      <p>Role: {user.role}</p>
      <button onClick={() => onUpdateRole(user.id, "admin")}>
        Promote to Admin
      </button>
    </div>
  );
}

```

REACT Component with Typed JS

src > UserProfileCard.tsx > GenericListProps

```tsx
import React from "react";

// 1. Strict Contract Interfaces
type UserRole = "admin" | "editor" | "viewer";

interface User {
  id: string;
  name: string;
  role: UserRole;
}

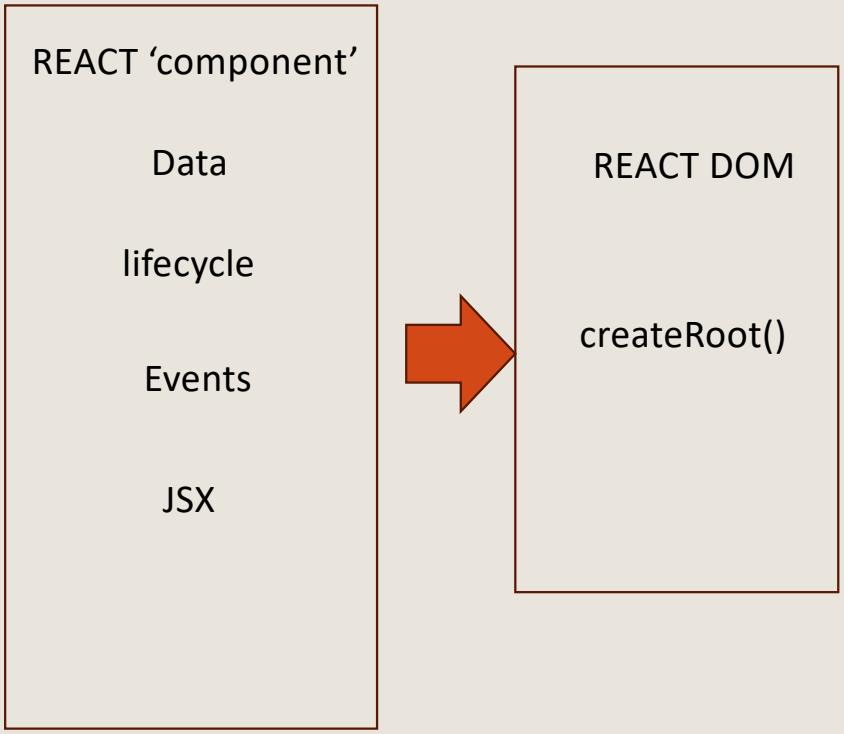
interface UserProfileCardProps {
  user: User;
  onUpdateRole: (userId: string, newRole: UserRole) => void;
}

// 2. Strongly Typed Component implementation
export function UserProfileCard({ user, onUpdateRole }: UserProfileCardProps) {
  return (
    <div className="profile-card">
      {/* TS validates that user.name is guaranteed to be a string */}
      <h2>{user.name.toUpperCase()}</h2>
      <p>Role: {user.role}</p>

      {/* TS enforces that 'admin' is a valid UserRole argument */}
      <button onClick={() => onUpdateRole(user.id, "admin")}>
        Promote to Admin
      </button>
    </div>
  );
}

```

REACT Component at a glance



- REACT function component
  - Parts of the page that are rendered by REACT DOM
- Data
  - Typed data
  - comes from somewhere and is rendered by the 'component'
- Lifecycle
  - Methods – e.g hooks – respond to component rendering process
- Events
  - Responding to user interactions
- JSX
  - Describe UI structures
- REACT DOM
  - Uses createRoot() from react-dom/client to enable concurrent rendering features and automatic batching.

Code a REACT APP

- Check library versions:

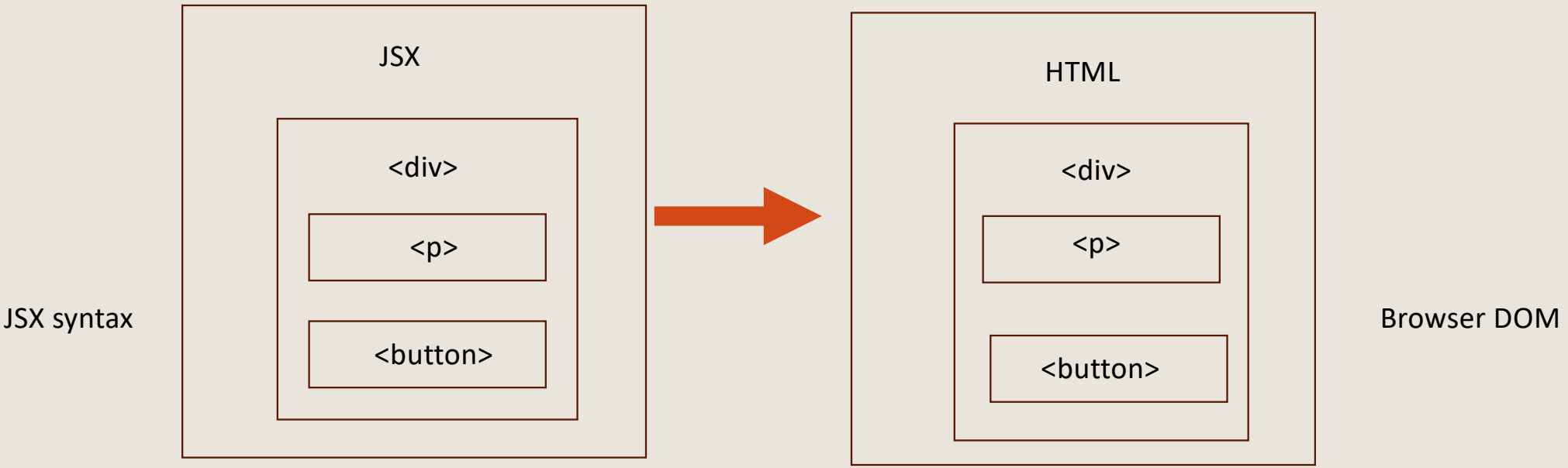
- Node –v
- Npm -v

- npm create vite@latest my-project -- --template react-ts

On Google Chrome - Navigate to http://localhost:5173/

JSX – JavaScript XML

- REACT component renders the content by returning some JSX
- JSX – HTML markups, mixed with custom tags for REACT ‘components’



JSX – JavaScript XML

```
6   return (
7     <>
8       <h1>Adv Web Development Setup</h1>
9     </>
10  );
```

- The return() method returns JSX and REACT DOM render it to the DOM node
- JSX
  - Must have a root / parent element

JSX root

Let's look at a simple example:

```
<div />
<div />
```

This gives us the following error:

```
Adjacent JSX elements must be wrapped in an enclosing tag
```

On the other hand, the following works:

```
<div>
  <div />
  <div />
</div>
```

```
6   return (
7     <>
8       <h1>Adv Web Development Setup</h1>
9     </>
10  );
```

- It is pretty annoying to have to add unnecessary ‘div’ tags just to make JSX work! Fortunately, we can use ‘<>’

JSX – rules

- Use lowercase for tag name
- Tag names are case sensitive
- Non-HTML elements are capitalised
- Can use any valid HTML tags as JSX tags – remember, they are case-sensitive!
  - <button> != <Button>

JSX – attributes

- JSX is not a standard language and that it gets transpiled into JavaScript. Because of this, some attributes cannot be used.
- instead of `class`, we have to use `className`, and instead of `for`, we have to use `htmlFor`:

```
<label className="awesome-label" htmlFor="name" />
```

JSX – Boolean

- For attribute without a value, JSX assumes its value is ‘true’, the same behaviour of the HTML

- This means that if we want to set an attribute to false, we have to declare it explicitly as false:

```
<button disabled />
React.createElement("button", { disabled: true });
```

JSX spread attributes (…)

- Handy way to pass all the attributes of a JavaScript object to an element
- The spread attributes operator, which comes from the Rest/Spread Properties for ECMAScript proposal https://github.com/tc39/proposal-object-rest-spread
- Avoid passing the entire JavaScript objects down to children by reference, but to use their primitive values
  - can be easily validated,
  - making components more robust,
  - error-proof

JSX spread attributes

```
3   function App() {
4     const foo = { id: "bar", className: "foo" };
5     return (
6       <>
7         <div {...foo}></div>
8       </>
9     );
10  }
11
```

```
<body> flex
  <div id="root"> == $0
    <div id="bar" class="foo"></div>
    <section> ... </section>
  </div>
  <script type="module" src="/src/main.tsx?t=1785771745257"></script>
</body>
</html>
```

JavaScript templating

- JavaScript expressions can be used as attributes values by enclosing them within curly braces:



Multi-line

```
<div>
  <Header />
  <div>
    <Main content={...} />
  </div>
</div>
```



```
<div>
  <Alert>{message}</Alert>
  <Button>Close</Button>
</div>
```



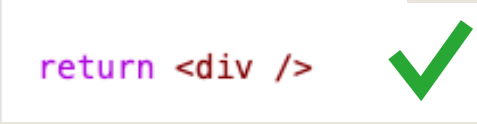
```
<div><Header /><div><Main content={...} /></div></div>
```



Multi-line

- Always remember to wrap your elements inside parentheses when you write them in multiple lines

```
7
8
9
return <div />
```



```
7
8
9
return
<div />
```



```
7
8
9
return (
<div />
)
```



```
6
7
8
9
10
11
return (
  <>
    <div {...foo} />
    <button disabled={disabled} />
  </>
);
```

Multi-properties

- Writing all the attributes on the same line, but this would lead to very long lines, which we don't want.

```
<button foo="bar" veryLongPropertyName="baz" onSomething={handleSomething} />
```

- Common practice -
  - write each attribute on a new line, with one level of indentation
  - then align the closing bracket with the opening tag

```
18    <button
19      foo="bar"
20      veryLongPropertyName="baz"
21      onSomething={handleSomething}
22    />
```

Conditionals

- Render some components only when certain conditions are matched
- Suppose we want to show a logout button only if the user is currently logged into our application.

```
let button
if (isLoggedIn) {
  button = <LogoutButton />
}
return <div>{button}</div>
```

Conditionals

```
let button
if (isLoggedIn) {
  button = <LogoutButton />
}
return <div>{button}</div>
```

- This works, but it is not very readable, especially if there are multiple components and multiple conditions.

Conditionals

```
<div>
  {isLoggedIn && <LoginButton />}
</div>
```

- This works because if the condition is false, nothing gets rendered, but if the condition is true, the ‘LoginButton’ JSX makes a call to React.createElement(LoginButton, null) and the element is returned to compose the resulting tree.

Conditionals

- If the condition has an alternative, (the classic if...else statement), and we want, for example, to show a logout button if the user is logged in and a login button otherwise, we can use JavaScript's if...else, as follows

```
let button
if (isLoggedIn) {
  button = <LogoutButton />
} else {
  button = <LoginButton />
}
return <div>{button}</div>
```

Conditionals

- Better still, we use a ternary condition, which makes the code more compact:

```
<div>
  {isLoggedIn ? <LogoutButton /> : <LoginButton />}
</div>
```

Conditionals

- More complicated example:

```
<div>
 {dataIsReady && (isAdmin || userHasPermissions) &&
   <SecretData />
 }
</div>
```

- It is clear that using the inline condition is a good solution, but the readability is strongly impacted!

Conditionals

- Solution: we can create a helper function inside our component and use it in JSX to verify the condition:

```
3
4   // 1. Define the props interface
5   interface AdminProps {
6     dataIsReady: boolean;
7     isAdmin: boolean;
8     userHasPermissions: boolean;
9   }
```

```
6   const Admin: React.FC<AdminProps> = ({
7     dataIsReady,
8     isAdmin,
9     userHasPermissions
10  }) => {
11    const canShowSecretData = dataIsReady && (isAdmin || userHasPermissions);
12    return <div>{canShowSecretData && <SecretData />}</div>;
13  };
14
```

C

- inside our component and

first-project > src > SecretData.tsx > ...

```tsx
import React from "react";

const SecretData = () => {
  return <div>SecretData</div>;
};

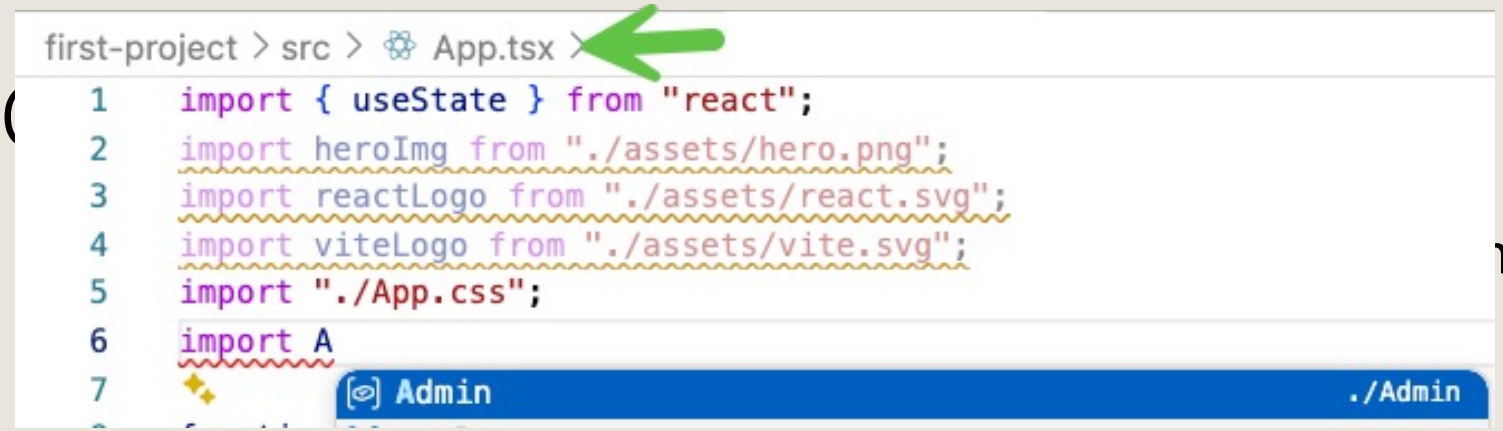
export default SecretData;

```

first-project > src > Admin.tsx > ...

```tsx
import Sec
```

SecretData ./SecretData



mponent and

```
13  const adminProps = {
14    dataIsReady: true,
15    isAdmin: true,
16    userHasPermissions: true
17  };
```

```
24      <Admin {...adminProps} />
```

Loops

- A very common operation in UI development is to display lists of items.
- When it comes to showing lists, using JavaScript as a template language is a very good idea.
- If we write a function that returns an array inside our JSX template, each element of the array gets compiled into an element.
- We can use any JavaScript expressions inside curly braces and the most common way to generate an array of elements, given an array of objects, is to use ‘map’ function.

Loops

- To create an unordered list to show the users, you can do the following:

```
<ul>
  {users.map(user =><li>{user.name}</li>)}
</ul>
```

Try it!

Create a user list

Render this list

Recall First-class functions

- In JavaScript, functions are first-class objects, which means that they can be assigned to variables and passed as parameters to other functions.

```
const add = (x, y) => x + y
```

Purity

- This concept very often in the React ecosystem, especially if you look into libraries such as Redux.
- What does it mean for a function to be pure?
- A function is pure when there are no side effects –
  - the function does not change anything that is not local to the functions itself.
  - a function that changes the state of an application, or modifies variables defined in the upper scope, or a function that touches external entities, such as the DOM, is considered impure.

Pure

```
const add = (x, y) => x + y
```

- It can be run multiple times, always getting the same result, because nothing is stored anywhere and nothing gets modified.

Not Pure

```
let x = 0
const add = y => (x = x + y)
```

- Running add(1) twice, we get two different results.
- The first time we get 1, but the second time we get 2, even if we call the same function with the same parameter.
- The reason we get that behaviour is that the global state gets modified after every execution.

Immutability

- What if we need to change the value of a variable?
- instead of changing the value of a variable, creates a new variable with a new value and returns it.
- This way of working with data is called immutability.
- An immutable value is a value that cannot be changed.

Immutability

```
const add3 = arr => arr.push(3)
const myArr = [1, 2]
add3(myArr) // [1, 2, 3]
add3(myArr) // [1, 2, 3, 3]
```

- doesn't follow immutability because it changes the value of the given array. Again, if we call the same function twice, we get different results.

```
const add3 = arr => arr.concat(3)
const myArr = [1, 2]
const result1 = add3(myArr) // [1, 2, 3]
const result2 = add3(myArr) // [1, 2, 3]
```

- using ‘concat’, which returns a new array without modifying the given one.
- After we have run the function twice, ‘myArr’ still has its original value.

Currying

- Currying is the process of converting a function that takes multiple arguments into a function that takes one argument at a time, returning another function.

```
JS currying.js > ...
1   const add = (x, y) => x + y;
2
```



```
4   const add = (x) => (y) => x + y;
5
```

```
PROBLEMS   OUTPUT   TERMINAL   ···   zsh
(base) jeannette@MacBook-Pro intro % node currying.js
[Function (anonymous)]
3
4
(base) jeannette@MacBook-Pro intro %
```

```
5   const add1 = add(1);
6   console.log(add1);
7   console.log(add1(2));
8   console.log(add1(3));
9
```

Composition

- Functions (and components) can be combined to produce new functions with more advanced features and properties.

```
JS composition.js > ...
1   const add = (x, y) => x + y;
2   const square = (x) => x * x;

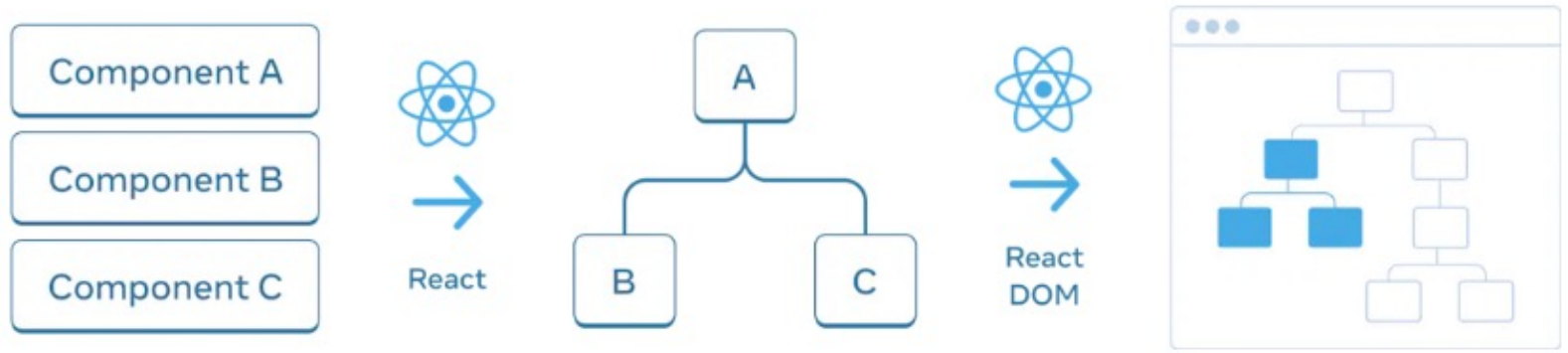
4   const addAndSquare = (x, y) => square(add(x, y));
5

6   console.log(addAndSquare(5, 5));
7
```

```
PROBLEMS  OUTPUT  TERMINAL  ···  zsh
(base) jeannette@MacBook-Pro intro % node composition.js
100
(base) jeannette@MacBook-Pro intro %
```

Your UI as a Tree

- Trees are a relationship model between items
- UI is often represented using tree structures



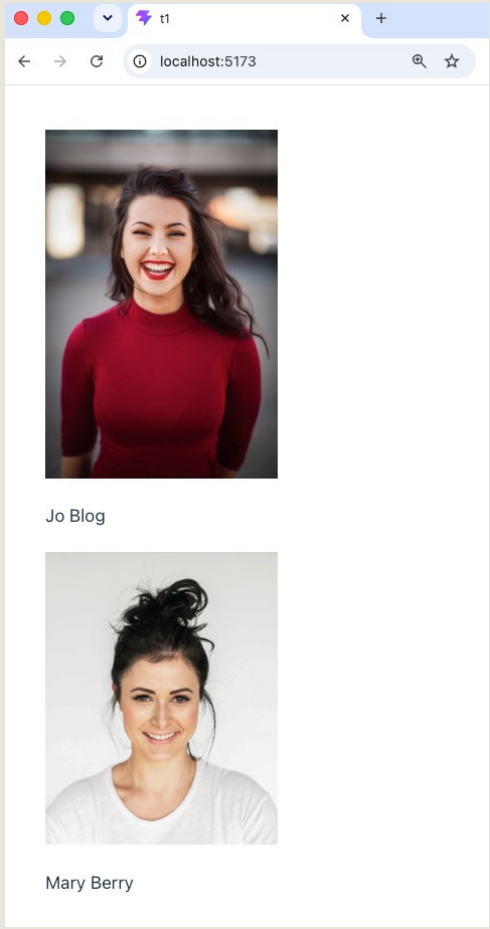
React creates a UI tree from your components. In this example, the UI tree is then used to render to the DOM.

Props

- React components use props to communicate with each other.
- Every parent component can pass some information to its child components by giving them props.
- Props might remind you of HTML attributes, but you can pass any JavaScript value through them, including objects, arrays, and functions.
- Props are the information that you pass to a JSX tag.
  - For example - className, src, alt, width, and height are some of the props you can pass to an <img>

Pass props to the child component

```
15    return (
16      <>
17        <div>
18          <Avatar person={{ ...Jo }} size={600} />
19          <p> {Jo.name} </p>
20        </div>
21        <div>
22          <Avatar person={{ ...Mary }} size={600} />
23          <p> {Mary.name} </p>
24        </div>
25      </>
26    );
27  };
28
```



Read props inside the child component

```tsx
const Avatar: React.FC<AvatarProps> = ({ person, size }) => {
  return (
    <img
      className="avatar"
      src={getImageUrl(person, size)}
      alt={person.name}
      width={size}
      height={size}
    />
  );
};
export default Avatar;
```

```tsx
// 1. Define the shape of the Person object
export interface Person {
  name: string;
  imageId: string;
}

// 2. Define the props for the Avatar component
interface AvatarProps {
  person: Person;
  size: number;
}
```

```tsx
// Helper function extracted outside the component so it isn't re-created on every render
const getImageUrl = (person: Person, size: number): string => {
  return `https://images.unsplash.com/photo-${person.imageId}?w=${size}`;
};
```

Read props inside the child component

- Don’t miss the pair of { and } curlies inside of (and) when declaring props:

```
function Avatar({ person, size }) {
  // ...
}
```

- This syntax is called “destructuring” and is equivalent to reading properties from a function parameter:

```
function Avatar(props) {
  let person = props.person;
  let size = props.size;
  // ...
}
```

Recall JSON

- JSON === string
- The values can be converted into JS object
- {}
- Key and value pairs
- Can have [] of {}

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/JSON



```
export const data = {
  kind: "books#volumes",
  totalItems: 3823,
  items: [
    {
      kind: "books#volume",
      id: "S85NCwAAQBAJ",
      etag: "PpRTmD2S0mk",
      selfLink: "https://www.googleapis.com/books/v1/volumes/S85NCwAAQBAJ",
      volumeInfo: {
        title: "It",
        subtitle: "A Novel",
        authors: ["Stephen King"],
        publisher: "Simon and Schuster",
        publishedDate: "2016-01-05",
        description:
          "In 1985, six men and one woman are called back together to search for a creature of
        industryIdentifiers: [
          {
            type: "ISBN_13",
            identifier: "9781501142970",
          },
          {
            type: "ISBN_10",
            identifier: "1501142976",
          },
        ],
        readingModes: {
          text: false,
          image: false,
        },
        pageCount: 1184,
        printType: "BOOK",
        categories: ["Fiction"],
        averageRating: 4,
        ratingsCount: 16,
        maturityRating: "NOT_MATURE",
        allowAnonLogging: false,
        contentVersion: "0.7.2.0.preview.0",
        panelizationSummary: {
          containsEpubBubbles: false,
          containsImageBubbles: false,
        },
        imageLinks: {
          smallThumbnail:
            "http://books.google.com/books/content?id=S85NCwAAQBAJ&printsec=frontcover&img=1&zo
          thumbnail:
            "http://books.google.com/books/content?id=S85NCwAAQBAJ&printsec=frontcover&img=1&zo
        },
        language: "en",
        previewLink:
          "http://books.google.co.uk/books?id=S85NCwAAQBAJ&printsec=frontcover&dq=stephen+king&
        infoLink:
          "http://books.google.co.uk/books?id=S85NCwAAQBAJ&dq=stephen+king&hl=&source=gbs_api",
        canonicalVolumeLink:
          "https://books.google.com/books/about/It.html?hl=&id=S85NCwAAQBAJ",
      },
      saleInfo: {
        country: "GB",
        saleability: "NOT_FOR_SALE",
        isEbook: false,
      },
```

Working with JSON

- The code:

src > BookList.tsx >

```tsx
import { data } from "./bookdata";
const { items } = data;

const BookList = () => {
  return (
    <section>
      <h2>BookList</h2>
      <div>
        {items.map((item) => {
          return <p>{item.id}</p>;
        })}
      </div>
    </section>
  );
};

export default BookList;
```

```tsx
export const data: BooksApiResponse = {
  kind: "books#volumes",
  totalItems: 3823,
  items: [
    {
      kind: "books#volume",
      id: "S85NCwAAQBAJ",
      etag: "PpRTmD2S0mk",
      selfLink: "https://www.googleapis.com/books/v1/volumes/S85NCwAAQBAJ",
      volumeInfo: {
        title: "It",
        subtitle: "A Novel",
        authors: ["Stephen King"],
        publisher: "Simon and Schuster",
        publishedDate: "2016-01-05",
        description:
          "In 1985, six men and one woman are called back together to search
        industryIdentifiers: [
          {
```

Working wit

- The code:

src > BookList.tsx >

```
1   import { data } fro
2   const { items } = d
3
4   const BookList = ()
5     return (
6       <section>
7         <h2>BookList<
8         <div>
9           {items.map(
10            return <p
11          })}
12        </div>
13      </section>
14    );
15  };
16
17  export default Book
18
```

t1

localhost:5173

BookList

S85NCwAAQBAJ

2BcECkYB7uoC

Qbm5ejloY5sC

LC32OW8iVUQC

3wZaEAAAQBAJ

wa6WDwAAQBAJ

PPNaAAAAMAAJ

XSWeBAAAQBAJ

L2sFzeynylUC

giKlEAAAQBAJ

```
e = {

eapis.com/books/v1/volumes/S85NCwAAQBAJ",

ster",
",

e woman are called back together to search
```

Event Handling in REACT

Vanilla JS

- addEventListner()
- Pass in the handler
- The script is either written in an external file or embedded in HTML document

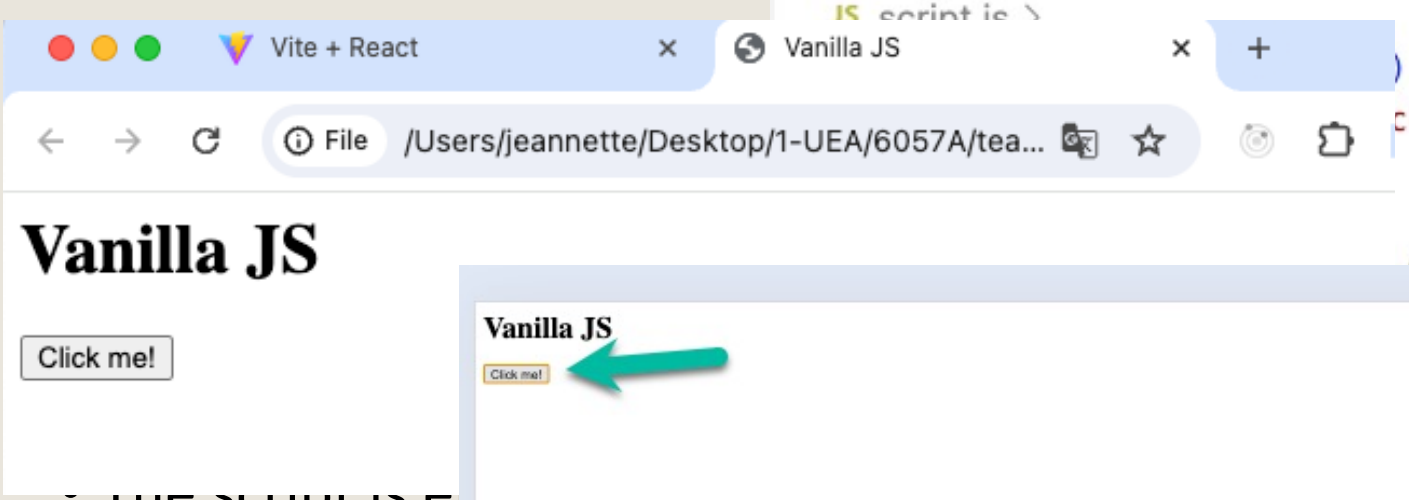
JS script.js > ...

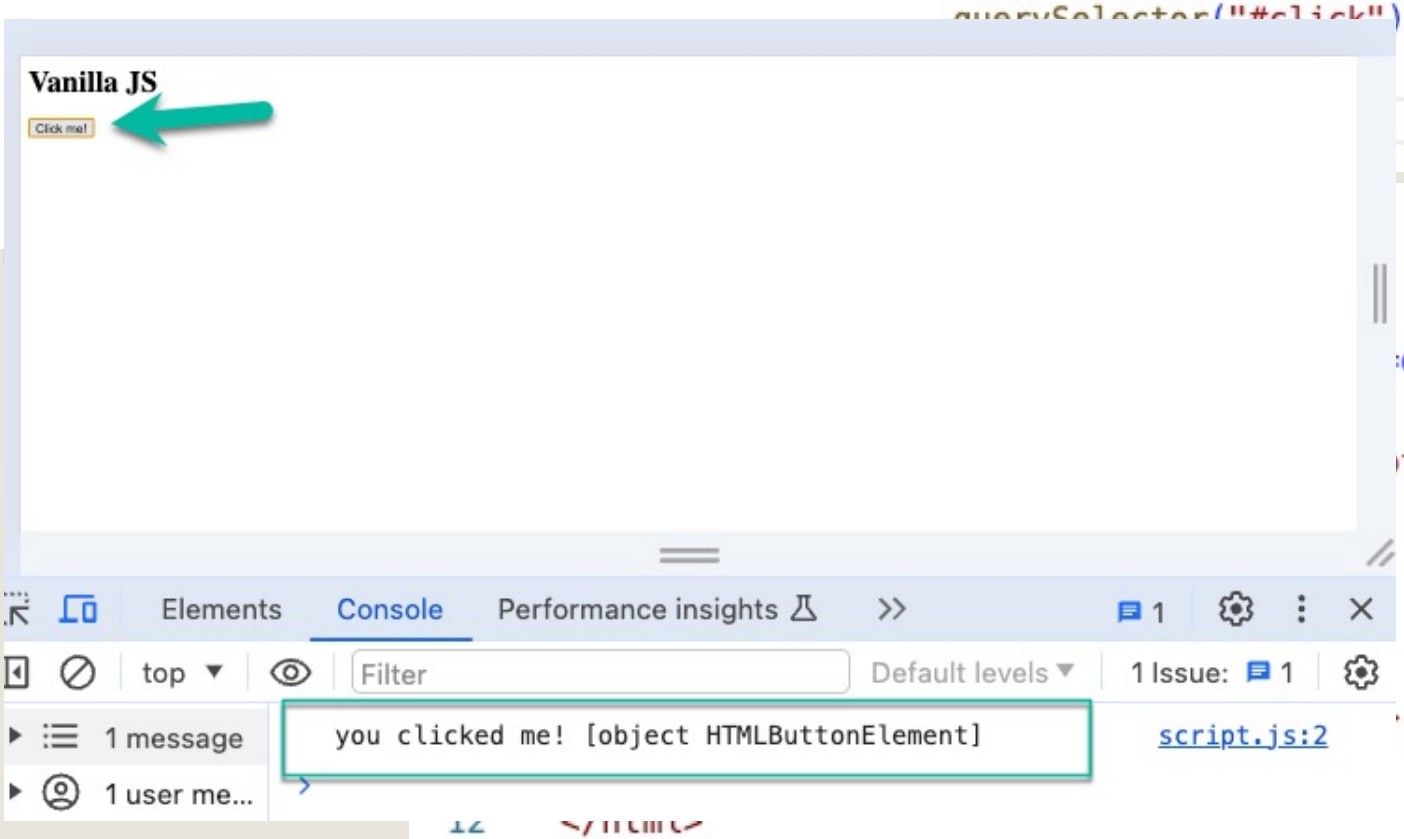
```
1   function handleClick(e) {
2     console.log(`you clicked me! ${e.target}`);
3   }
4
5   const click = document.querySelector("#click");
6   click.addEventListener("click", handleClick);
7
```

<> index.html > ...

```
1   <html>
2     <head>
3       <meta name="viewport" content="width=dev
4       <title>Vanilla JS</title>
5       <script src="script.js" defer></script>
6     </head>
7
8     <body>
9       <h1>Vanilla JS</h1>
10      <button id="click">Click me!</button>
11    </body>
12  </html>
13
```

- The script is e
  in an external
  embedded in
  document





In REACT

- We will first define handler function
- then pass it as a prop to the appropriate JSX tag
- onClick()

src > Event.tsx > ...

```tsx
import React from "react";

const Events = () => {
  const handleClick = (e: React.MouseEvent<HTMLButtonElement>) => {
    console.log(`the id you clicked --> ${e.currentTarget.id}`);
    console.log(`you clicked me!  ${e.target}`);
  };
  return (
    <div>
      <h2>Events</h2>
      <button id="click" onClick={handleClick}>
        {" "}
        Click me!{" "}
      </button>
    </div>
  );
};

export default Events;
```



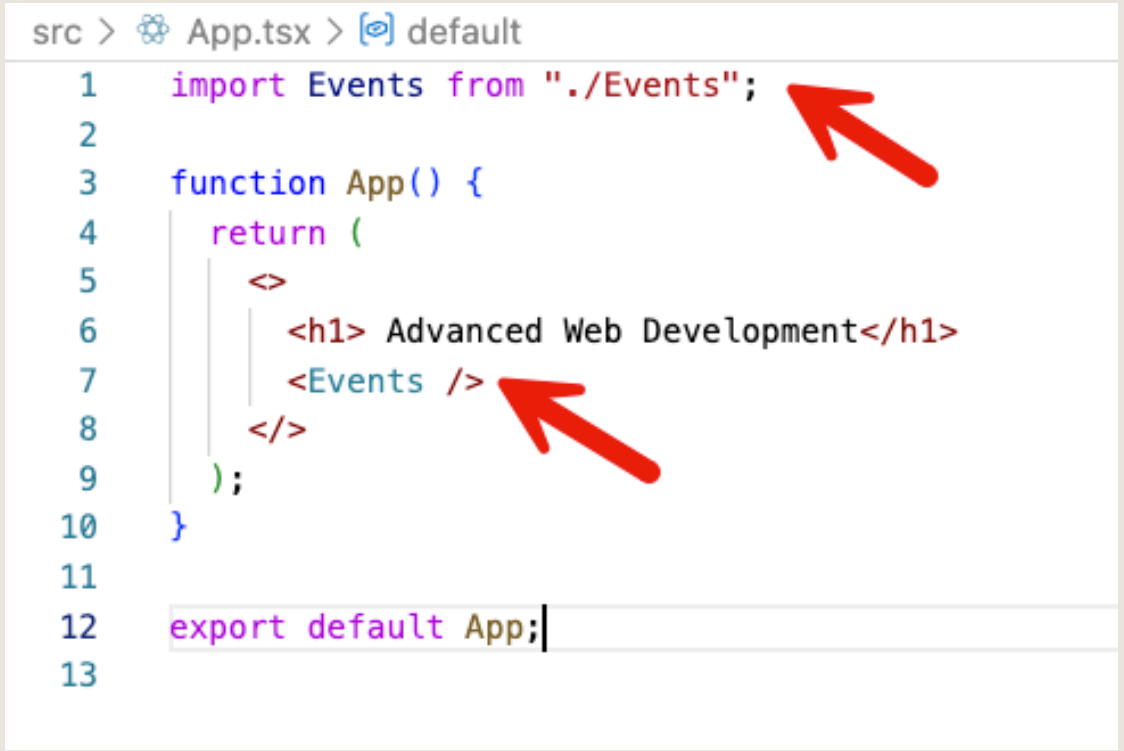
Clone the repo if you wish!

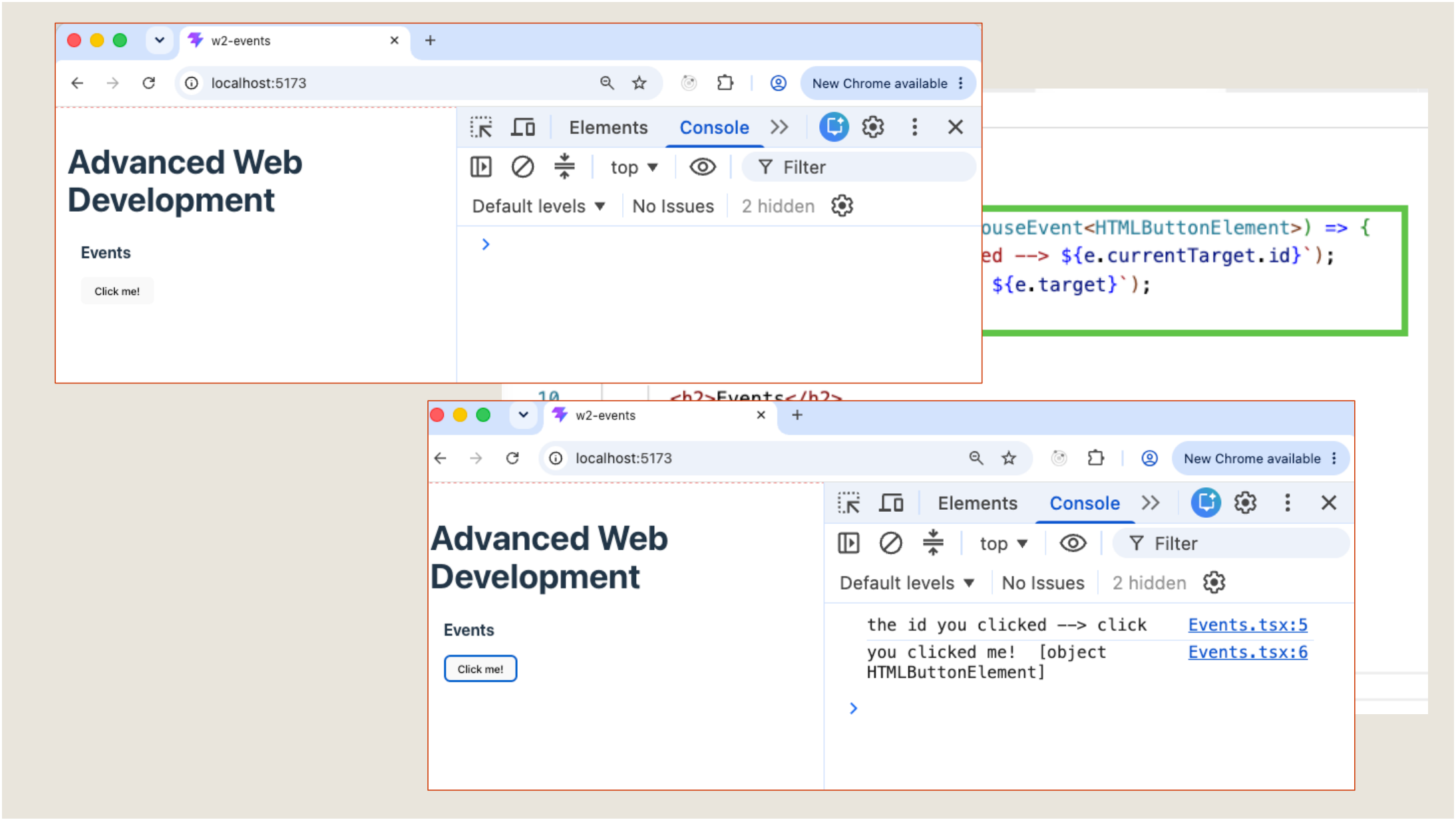
git clone

https://stugit.cmp.uea.ac.uk/jc/Adv-WebDev-26-27-W2-Events.git

git clone https://github.com/AdvWebDev2026-27/w2-events.git

- Clone the repo
- Type “npm install”
- Go to App.jsx , comment out everything apart from Event
- To bring up the server type: “npm run dev”
- Navigate to http://localhost:5173/




w2-events
localhost:5173
New Chrome available
Advanced Web Development
Events
Click me!
Elements
Console
top
Filter
Default levels
No Issues
2 hidden
ouseEvent<HTMLButtonElement>) => {
ed --> ${e.currentTarget.id}`);
${e.target}`);
w2-events
localhost:5173
New Chrome available
Advanced Web Development
Events
Click me!
Elements
Console
top
Filter
Default levels
No Issues
2 hidden
the id you clicked --> click
Events.tsx:5
you clicked me! [object HTMLButtonElement]
Events.tsx:6

React Events in TypeScript

Synthetic Event Types:

React events use specialised wrappers. Common ones include:

React.MouseEvent<HTMLButtonElement> (Clicks)

React.ChangeEvent<HTMLInputElement> (Input changes)

React. SubmitEvent<HTMLFormElement> (Form submissions)

e.target vs e.currentTarget

e.currentTarget refers directly to the element receiving the event listener (the <button>), so TypeScript knows its exact DOM type.

e.target refers to the element that triggered the click (which could be a nested inside the button), so TS types it as a generic EventTarget.

Responding to Events

- Functions passed to event handlers must be passed, not called.

passing a function (correct)	calling a function (incorrect)
`<button onClick={handleClick}>`	`<button onClick={handleClick()}>`

passing a function (correct)	calling a function (incorrect)
`<button onClick={() => alert('...')}>`	`<button onClick={alert('...')}>`

Reading props in event handlers

- Because event handlers are declared inside of a component, they have access to the component's props

src > AlertButton.tsx > ...

```tsx
import type { ReactNode } from "react";

// 1. Define the props contract interface
interface AlertButtonProps {
  message: string;
  children: ReactNode; // Supports JSX tags, text, numbers, or fragmen
}

const AlertButton = ({ message, children }: AlertButtonProps) => {
  const handleClick = (): void => {
    alert(message);
  };

  return <button onClick={handleClick}>{children}</button>;
};

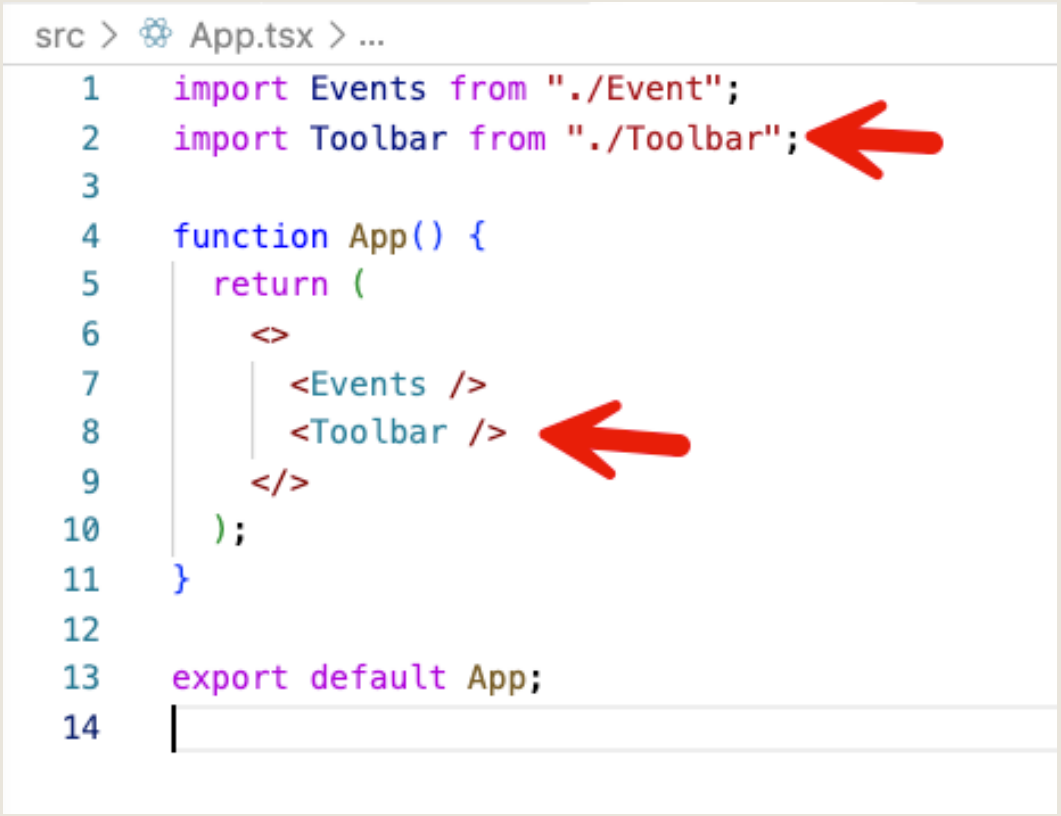
export default AlertButton;
```

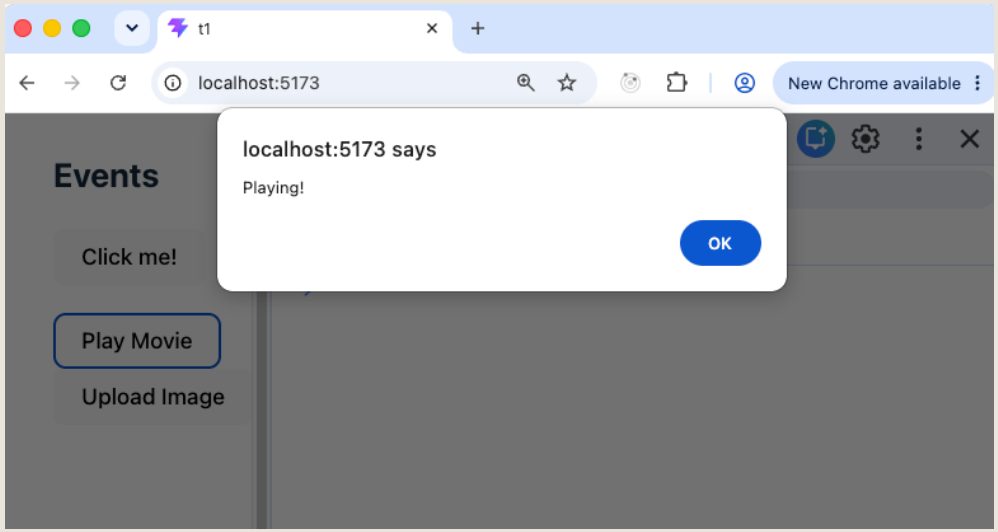
```tsx
  return (
    <button
      onClick={() => {
        alert(message);
      }}
    >
      {children}
    </button>
  );
};
```

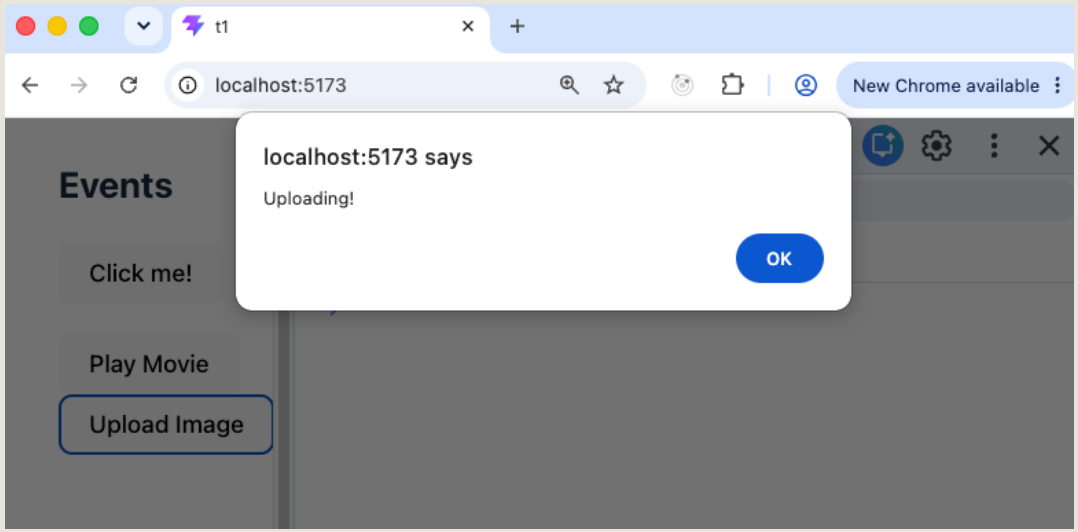
Responding to Events

- Make sure that you use the appropriate HTML tags for your event handlers.
- For example, to handle clicks, use <button onClick={handleClick}> instead of .

To try it yourself - go to VSCode

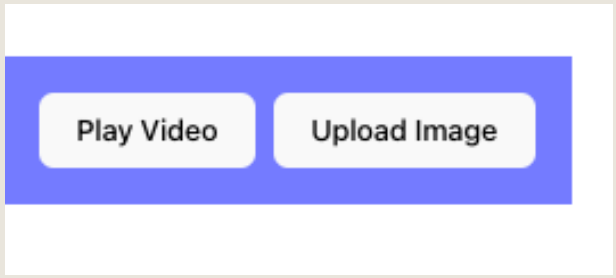



t1
localhost:5173
New Chrome available
Events
Click me!
Play Movie
Upload Image
localhost:5173 says
Playing!
OK


t1
localhost:5173
New Chrome available
Events
Click me!
Play Movie
Upload Image
localhost:5173 says
Uploading!
OK

Event Challenge

- Which handler do you think will fire when you click a button?



src > ToolbarChallenge.tsx > ...

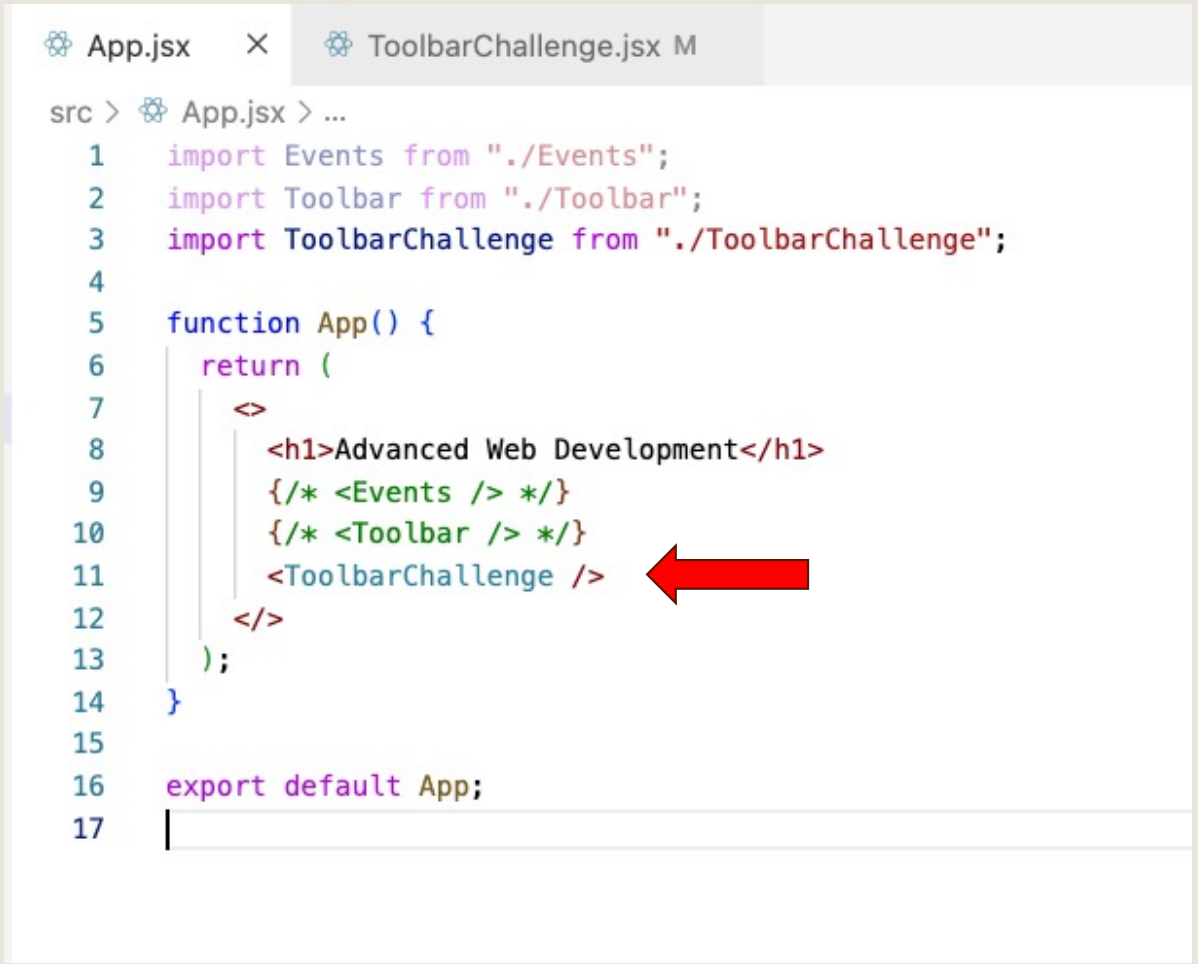
```tsx
import React from "react";

const ToolbarChallenge = () => {
  const handleToolbar = () => {
    console.log("You clicked toolbar");
  };
  const handleVideo = (e: React.MouseEvent<HTMLButtonElement>) => {
    console.log("You clicked Video");
  };
  const handleImage = (e: React.MouseEvent<HTMLButtonElement>) => {
    console.log("You clicked image");
  };

  const handleSubmit = (e: React.SubmitEvent<HTMLFormElement>) => {
    e.preventDefault();
    console.log("on submit");
  };
  return (
    <div className="Toolbar" onClick={handleToolbar}>
      <form onSubmit={handleSubmit}>
        <button onClick={handleVideo}>Play Video</button>
        <button onClick={handleImage}>Upload Image</button>
      </form>
    </div>
  );
};

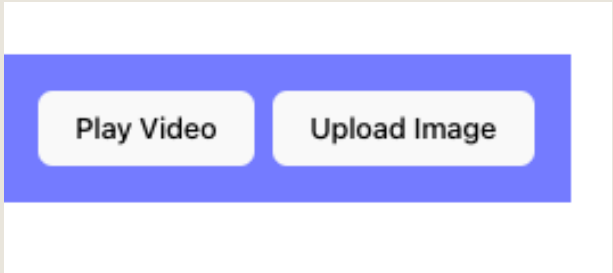
export default ToolbarChallenge;
```

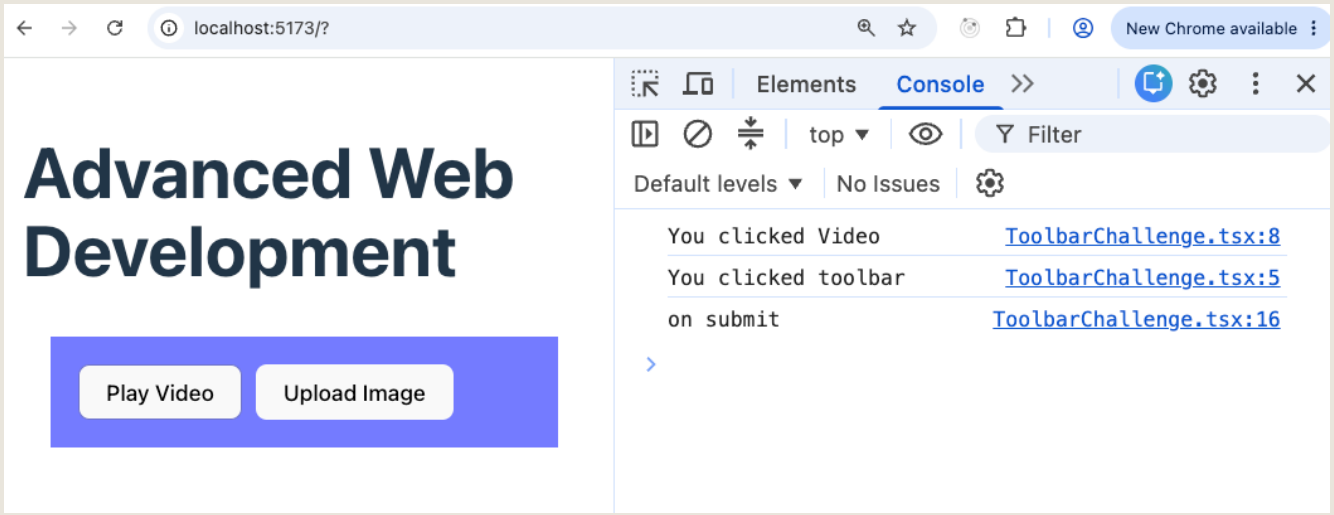
To try it yourself - go to VSCode



Event Challenge

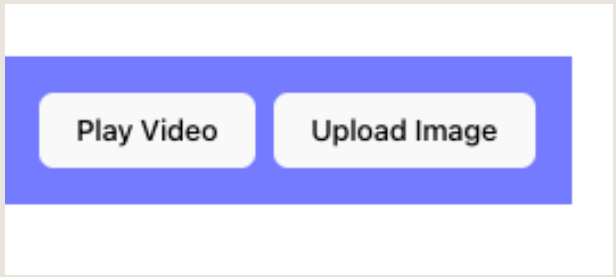
- Which handler do you think will fire when you click a button?

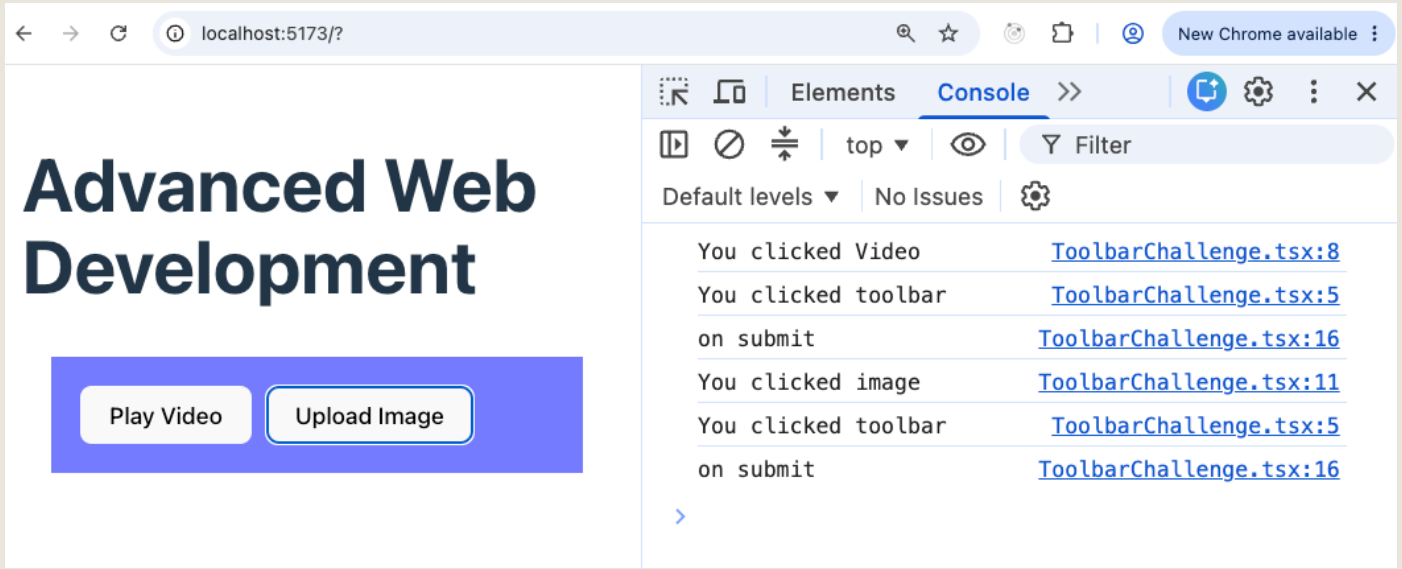




Event Challenge

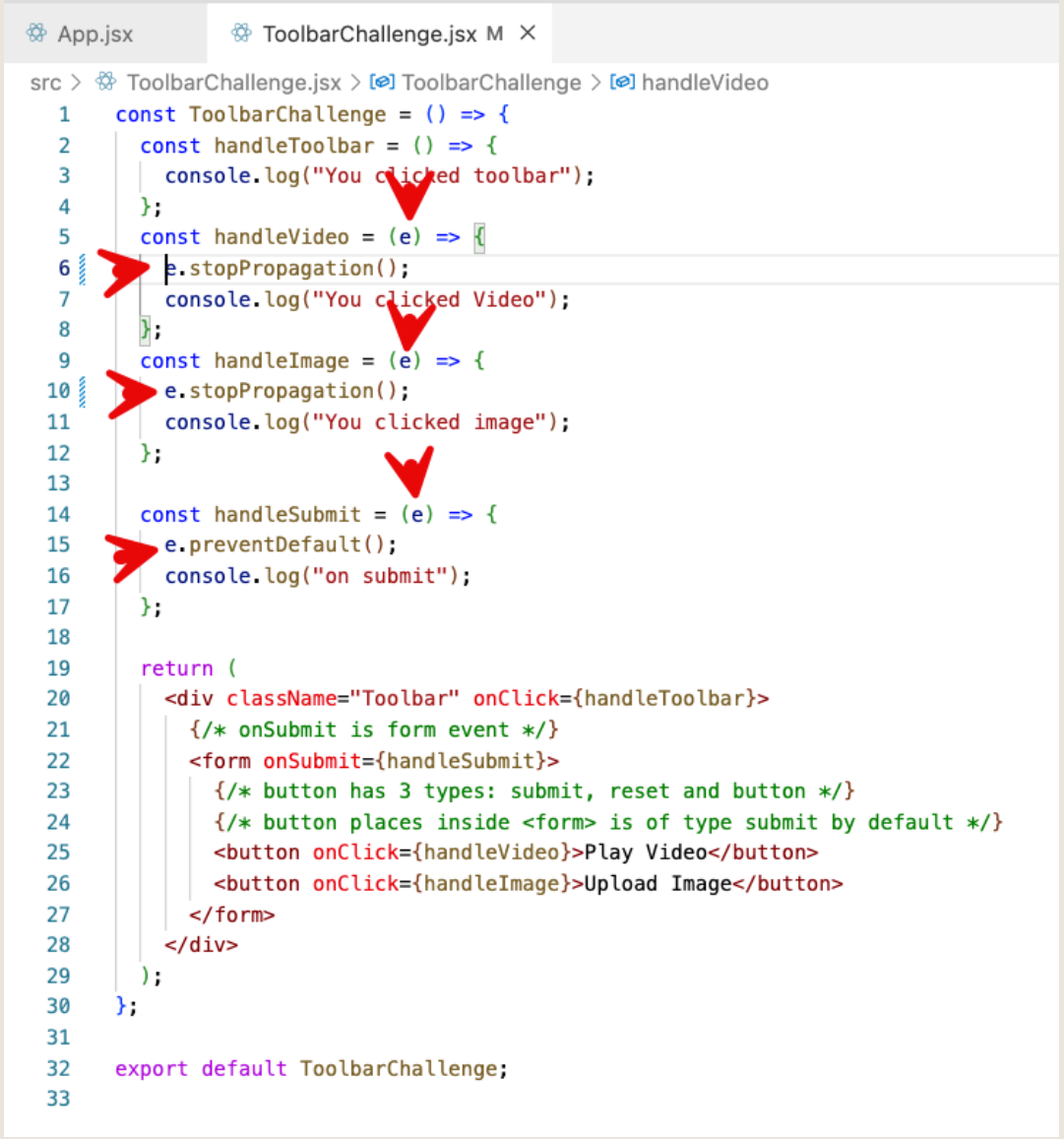
- Which handler do you think will fire when you click a button?





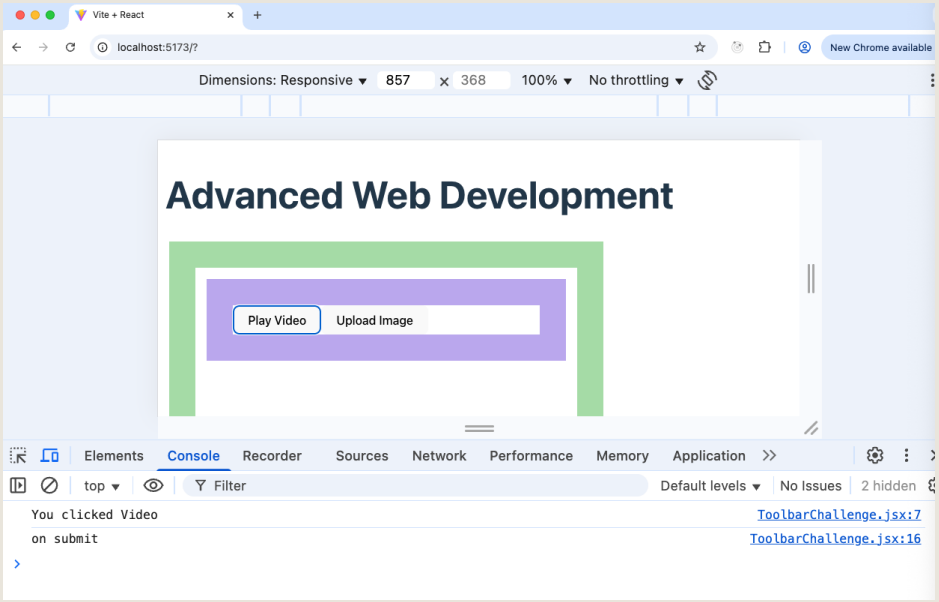
Stopping propagation

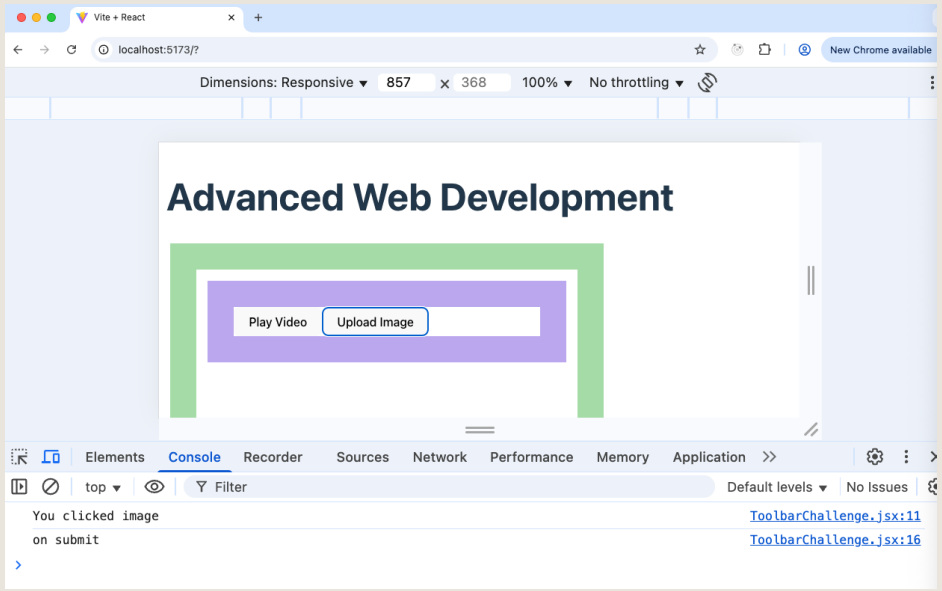
- To prevent an event from reaching parent components, you need to call e.stopPropagation()



Stopping propagation

- To prevent an event from reaching parent components, you need to call e.stopPropagation()





Preventing default behaviour - Form

- Some browser events have default behaviour associated with them. For example, a <form> submit event, which happens when a button inside of it is clicked, will reload the whole page by default.

```
16    const handleSubmit = (e) => {
17      e.preventDefault();
18    };
19    return (
20      <div className="Toolbar" onClick={handleToolbar}>
21        <form onSubmit={handleSubmit}>
22          <button onClick={handleVideo}>Play Video</button>
23          <button onClick={handleImage}>Upload Image</button>
24        </form>
25      </div>
26    );
27  };
```

REACT State

A component’s memory

REACT Rendering – 3 Steps

- Triggering a render
  - Initial render
    - Trigger by createRoot() with a target DOM node

```
6    createRoot(document.getElementById("root")).render(<App />);
7
```

  - A component state has been updated
    - Re-renders when state is updated
- Rendering the component
  - On initial render, React will call the root component.
  - For subsequent renders, React will call the function component whose state update triggered the render.
- Committing to the DOM
  - For the initial render, React will use the appendChild() DOM API to put all the DOM nodes it has created on screen.
  - For re-renders, REACT calculates changes and makes the DOM match with latest rendering output

useState hook

- A state variable to retain the data between renders.
- A state setter function to update the variable and trigger React to render the component again.

- To add a state variable, import ‘useState’ from React at the top of the file

```
1    import { useState } from "react";
```

useState hook

- useState hook always returns two items – the state and setter function

- So, we can create a state value by invoking the useState()

```
7   const [index, setIndex] = useState(0);
8
```

- Here, I set the initial state value for ‘index’ to ‘0’

To trigger the re-rendering

- Use the setter function in the handler:



- By convention, the name of the setter function starts with ‘set’

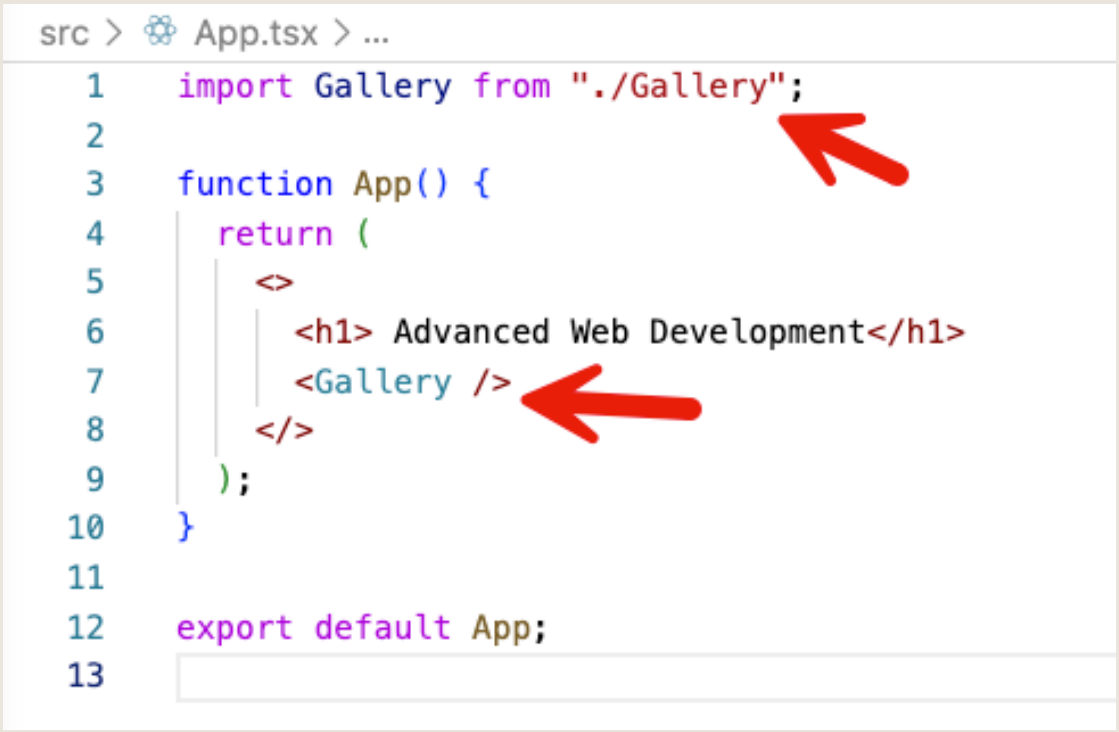
Clone the repo if you wish!

git clone

https://stugit.cmp.uea.ac.uk/jc/Adv-WebDev-26-27-W2-States.git

Follow my code:

git clone https://github.com/AdvWebDev2026-27/w2-states.git




t1
localhost:5173
Advanced Web Development
Next
Homenaje a la Neurocirugía by Marta Colvin Andrade
(1 of 12)
Show details


t1
localhost:5173
Advanced Web Development
Next
Floralis Genérica by Eduardo Catalano
(2 of 12)
Show details

useState hook

- You can have as many state variables of as many types as you like in one component.

```
// useState types are automatically inferred as number and boolean
const [index, setIndex] = useState<number>(0);
const [showMore, setShowMore] = useState<boolean>(false);

const handleMoreClick = (): void => {
  setShowMore((prevShow) => !prevShow);
};

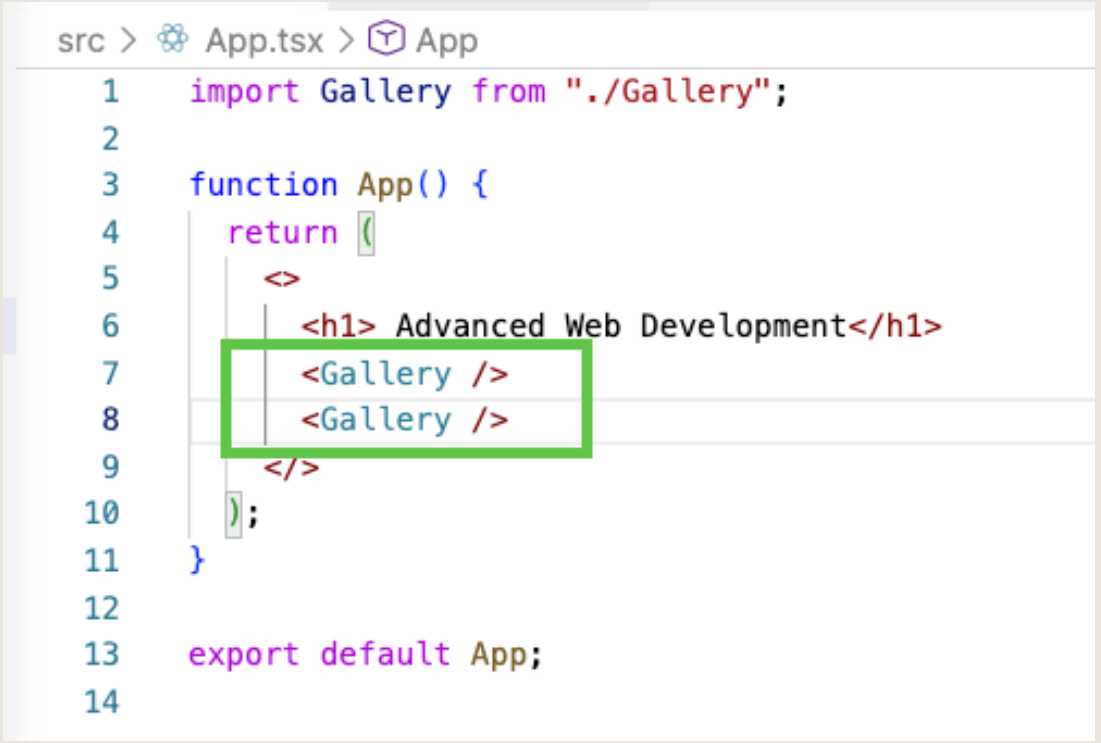
// Typed lookup with safety check or boundary wrapping
const sculpture: Sculpture = sculptureList[index];

  return (
    <section>
      <button onClick={handleClick}>Next</button>
      <h2>
        <i>{sculpture.name} </i>
        by {sculpture.artist}
      </h2>
      <h3>
        ({index + 1} of {sculptureList.length})
      </h3>
      <img src={sculpture.url} alt={sculpture.alt} />

      <button onClick={handleMoreClick}>
        {showMore ? "Hide" : "Show"} details
      </button>
      {showMore && <p>{sculpture.description}</p>}
    </section>
  );
};
```

useState hook

- State is local to a component instance on the screen.
- In other words, if you render the same component twice, each copy will have completely isolated state! Changing one of them will not affect the other.



t1

localhost:5173

Homenaje a la Neurocirugía by Marta Colvin Andrade

(1 of 12)



Show details

Next

Homenaje a la Neurocirugía by Marta Colvin Andrade

(1 of 12)



Show details

t1

localhost:5173

Next

Floralis Genérica by Eduardo Catalano

(2 of 12)



Show details

Next

Homenaje a la Neurocirugía by Marta Colvin Andrade

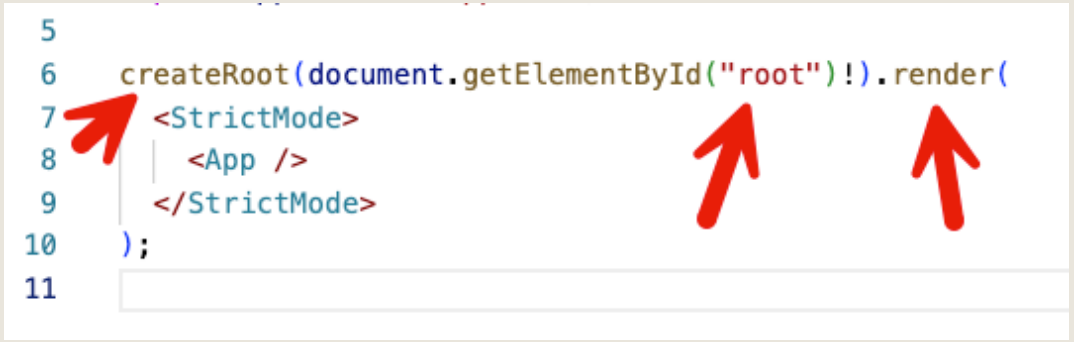
(1 of 12)



Show details

Render and Commit in 3 steps (Step 1)

- Step1 - Triggering a render – 2 reasons:
  - initial render on app starts by calling createRoot() with target DOM node



  - The component's (or one of its ancestors') state has been updated
    - Once the component has been initially rendered, you can trigger further renders by updating its state with the 'set' function.
    - Updating your component's state automatically queues a render.

Render and Commit in 3 steps (Step 2)

- Step2 - Rendering a component
  - After you trigger a render, React calls your components to figure out what to display on screen.
  - “Rendering” is React calling your components.
- On initial render, React will call the root component.
- For subsequent renders, React will call the function component whose state update triggered the render.

Render and Commit in 3 steps (Step 3)

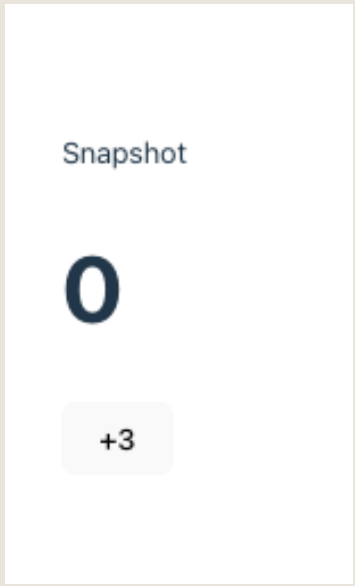
- Step3 - Committing to the DOM
  - After rendering (calling) your components, React will modify the DOM.
  - For the initial render, React will use the appendChild() DOM API to put all the DOM nodes it has created on screen.
  - For re-renders, React will apply the minimal necessary operations (calculated while rendering!) to make the DOM match the latest rendering output.
  - After rendering is done and React updated the DOM, the browser will render the screen.

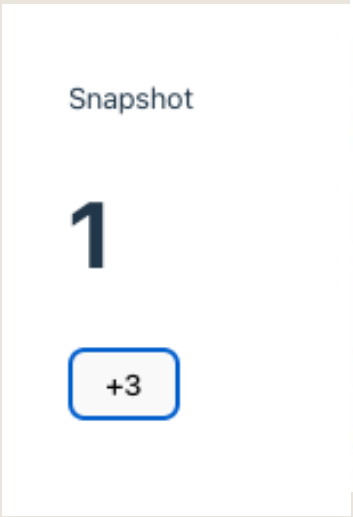
So far, we have learned

- Setting a state value triggers a re-render from React.
- This means that for an interface to react to the event, you need to update the state.

Let’s see another example

- What is the output?






```
3   const Snapshot = () => {
4     const [number, setNumber] = useState<number>(0);
5
6     const handleClick = () => {
7       setNumber(number + 1);
8       setNumber(number + 1);
9       setNumber(number + 1);
10    };
11
12    return (
13      <div>
14        <h2> Snapshot </h2>
15        <h3>{number}</h3>
16        <button onClick={handleClick}>+3</button>
17      </div>
18    );
19  };
20
```

Clone the repo if you wish!

git clone
https://stugit.cmp.uea.ac.uk/jc/Adv-WebDev-26-27-W2-Snapshot.git

Let’s see another example

- What is the output?
- that number only increments once per click!
- Setting state only changes it for the next render.
- During the first render, number was 0. and in that render’s onClick handler, the value of number is still 0 for each setNumber(number + 1) was called !

```
3   const Snapshot = () => {
4     const [number, setNumber] = useState(0);
5
6     const handleClick = () => {
7       setNumber(number + 1);
8       setNumber(number + 1);
9       setNumber(number + 1);
10    };
11    return (
12      <div>
13        Snapshot
14        <h1>{number}</h1>
15        <button onClick={handleClick}>+3</button>
16      </div>
17    );
```

Even though you called setNumber(number + 1) three times, in this render’s event handler number is always 0, so you set the state to 1 three times

What is the output on the alert?

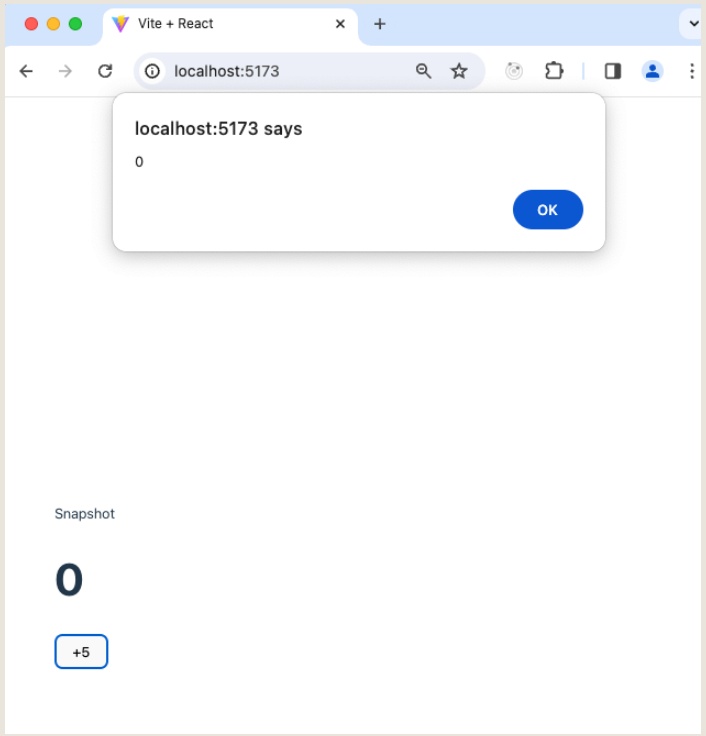
```
3   const Snapshot = () => {
4     const [number, setNumber] = useState(0);
5
11
12    const handleClick2 = () => {
13      setNumber(number + 5);
14      alert(number);
15    };
16
17    return (
18      <div>
19        Snapshot
20        <h1>{number}</h1>
21        <button onClick={handleClick2}>+5</button>
22      </div>
23    );
24  };
25
```



What is the output on the alert?

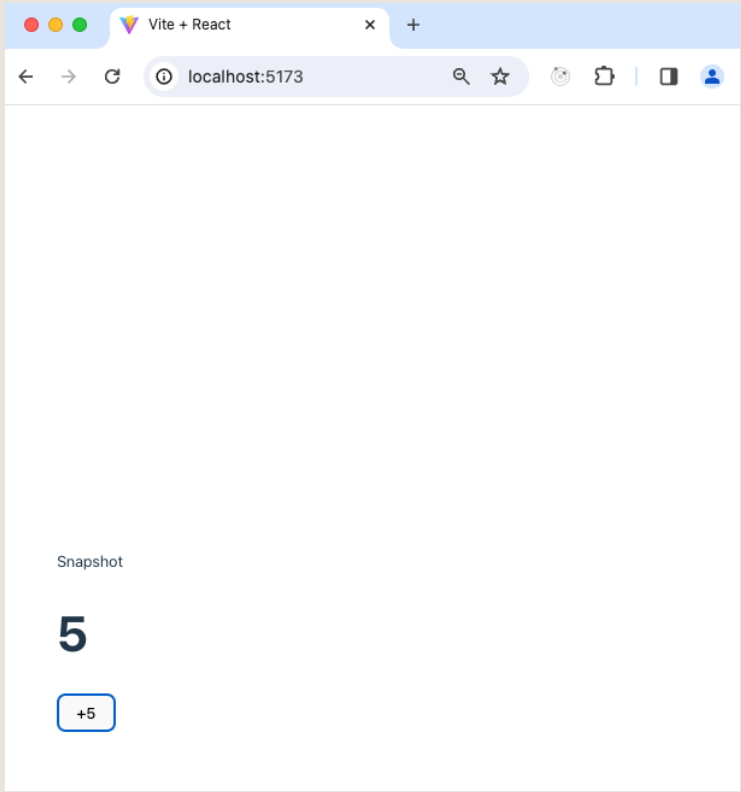
```
3   const Snapshot = () => {
4     const [number, setNumber] = useState(0);
5
```

```
12    const handleClick2 = () => {
13      setNumber(number + 5);
14      alert(number);
15    };
16
17    return (
18      <div>
19        Snapshot
20        <h1>{number}</h1>
21        <button onClick={handleClick2}>+5</button>
22      </div>
23    );
24  };
25
```



What is the output on the alert?

```
3   const Snapshot = () => {
4     const [number, setNumber] = useState(0);
5
11
12    const handleClick2 = () => {
13      setNumber(number + 5);
14      alert(number);
15    };
16
17    return (
18      <div>
19        Snapshot
20        <h1>{number}</h1>
21        <button onClick={handleClick2}>+5</button>
22      </div>
23    );
24  };
25
```



After re-rendering

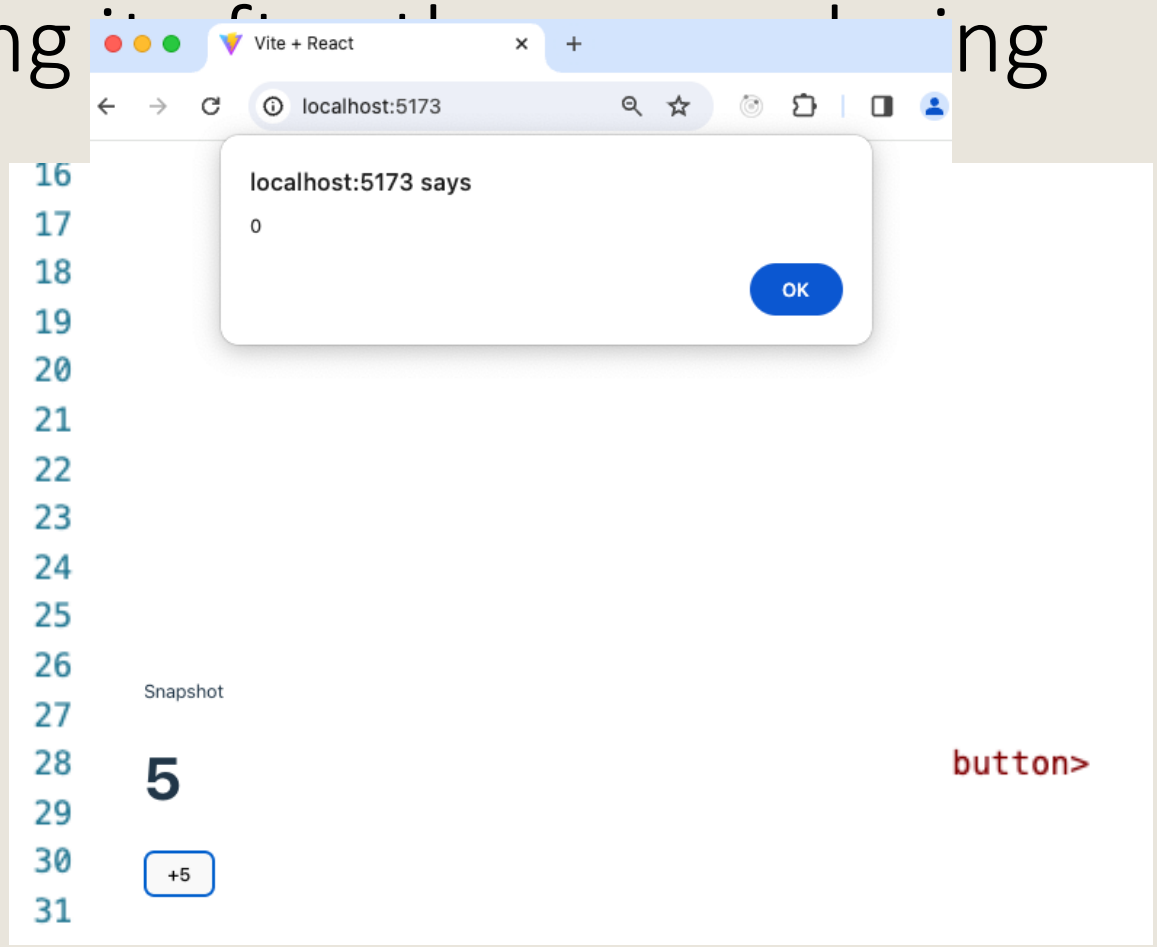
What about delaying it after the re-rendering

- What is the output on the alert?

```
16
17    const handleClick3 = () => {
18      setNumber(number + 5);
19      setTimeout(() => {
20        alert(number);
21      }, 3000);
22    };
23
24    return (
25      <div>
26        Snapshot
27        <h1>{number}</h1>
28        <button onClick={handleClick3}>+5</button>
29      </div>
30    );
31  };
```

What about delaying it after the re-rendering

- What is the output on the alert?
- The output is still 0 !
- A state variable's value never changes within a render, even if its event handler's code is asynchronous.
- React keeps the state values "fixed" within one render's event handlers. You don't need to worry whether the state has changed while the code is running.



What about if we want to update the state value before the next render?

- Solution – pass a function (called updater function) that calculates the next state based on the previous one in the queue, like setNumber(n => n + 1).

```
23
24    const handleClick4 = () => {
25      setNumber((n) => {
26        return n + 1;
27      });
28      setNumber((n) => {
29        return n + 1;
30      });
31      setNumber((n) => {
32        return n + 1;
33      });
34    };
35
36    return (
37      <div>
38        Snapshot
39        <h1>{number}</h1>
40        <button onClick={handleClick4}>+3</button>
41      </div>
42    );
43  };
44
```

updater function

- React queues this function to be processed after all the other code in the event handler has run.
- During the next render, React goes through the queue and gives you the final updated state.

```
24  const handleClick4 = () => {
25    setNumber((n) => {
26      return n + 1;
27    });
28    setNumber((n) => {
29      return n + 1;
30    });
31    setNumber((n) => {
32      return n + 1;
33    });
34  };
```

Here's how React works through these lines of code while executing the event handler:

setNumber(n => n + 1): n => n + 1 is a function. React adds it to a queue.
setNumber(n => n + 1): n => n + 1 is a function. React adds it to a queue.
setNumber(n => n + 1): n => n + 1 is a function. React adds it to a queue.

Updating Objects in State

- State can hold any kind of JavaScript value, including objects.
- We shouldn’t change objects that we hold in the React state directly.
- Instead, when we want to update an object, we need to create a new one (or make a copy of an existing one), and then set the state to use that copy.

Follow my code:

App.jsx M | Mutation.jsx M

src > App.jsx > ...

```jsx
import Snapshot from "./Snapshot";
import Mutation from "./Mutation";
import ShareState from "./ShareState";

function App() {
  return (
    <>
      <h1>Advanced Web Development</h1>
      <h2>Snapshot</h2>
      {/* <Snapshot /> */}
      <Mutation />
      {/* <ShareState /> */}
    </>
  );
}

export default App;
```

App.jsx M | Mutation.jsx M

src > Mutation.jsx > Mutation

```jsx
const Mutation = () => {
//   };

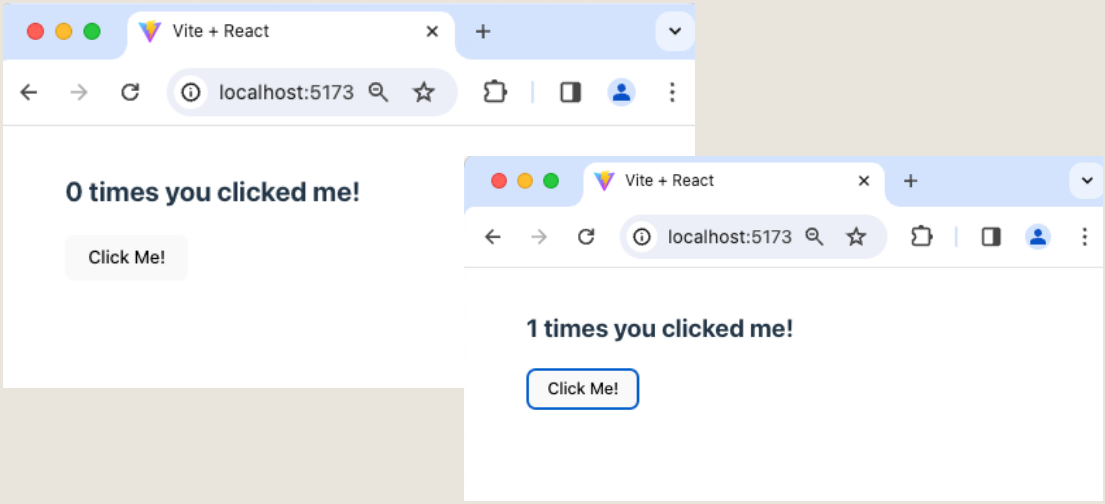
const handleClick = () => {
  setCount((count) => {
    return count + 1;
  });
};

return (
  <div>
    <h2>{count} times you clicked me! </h2>
    <button onClick={handleClick}>Click Me!</button>
    {/* <div>
      <label>
        {" "}
        Name
        <input onChange={handleUser} value={user.name} />
      </label>
    </div> */}
    {/* <div>
      <label>
```

Immutable

- Numbers, strings Booleans are immutable
  - Unchangeable
  - Read-only
- We can trigger a re-render to replace the value
- setCount()
  - the count state changed from its previous value + 1
  - But the previous value has not changed.

```
4   const [count, setCount] = useState(0);
5
6   const handleClick = () => {
7     setCount((count) => {
8       return count + 1;
9     });
10  };
```



Immutable Object?

- In REACT technically yes, but we should treat them as immutable
  - Unchangeable
  - Read-only
- Instead of mutating them, you should always replace them.

```
const [user, setUser] = useState<User>({
  name: "Jeannette Chin",
  email: "jc@test.com"
});

  const handleUser = () => {
    user.name = "Jo Blog";
  };
```



```
const [user, setUser] = useState<User>({
  name: "Jeannette Chin",
  email: "jc@test.com"
});

const handleUser = (e: React.ChangeEvent<HTMLInputElement>) => {
  setUser({
    ...user, // copy the old fields
    name: e.target.value /// overide this field
  });
  console.log(user);
};
```



Updating Arrays in State

- Arrays are mutable in JavaScript, but we should treat them as immutable when we store them in state.
- Just like with objects, when we want to update an array stored in state, we need to create a new one (or make a copy of an existing one), and then set state to use the new array.

- slice() (OK) VS splice() (mutate)
- push() – mutate array
- pop() – mutate array

Updating Arrays in State – how?

- Every time we want to update an array, we'll want to pass a new array to our state setting function.
- Then we can set our state to the resulting new array.

```
17  const [user, setUser] = useState<User>({
18    name: "Jeannette Chin",
19    email: "jc@test.com",
20  });
21  const [tickets, setTickets] = useState<Ticket[]>([]);
22  const addTicket = (e: React.MouseEvent<HTMLButtonElement>) => {
23    setTickets([
24      // new array
25      ...tickets, // contains old tickets
26      { id: unique_id, user }, // add one new item at the end
27    ]);
28  };
29
```

Updating Arrays in State – how?

- To render an array, use map()

```
<h3>Tickets</h3>
<ul>
  {tickets.map((ticket) => {
    return <li key={ticket.id}>{ticket.user.name}</li>;
  })}
</ul>
```

Follow my code:

src > App.tsx > ...

```jsx
import Snapshot from "./Snapshot";
import Mutation from "./Mutation";

function App() {
  return (
    <>
      <h1> Advanced Web Development</h1>

      {/* <Snapshot /> */}
      <Mutation />
    </>
  );
}

export default App;

```

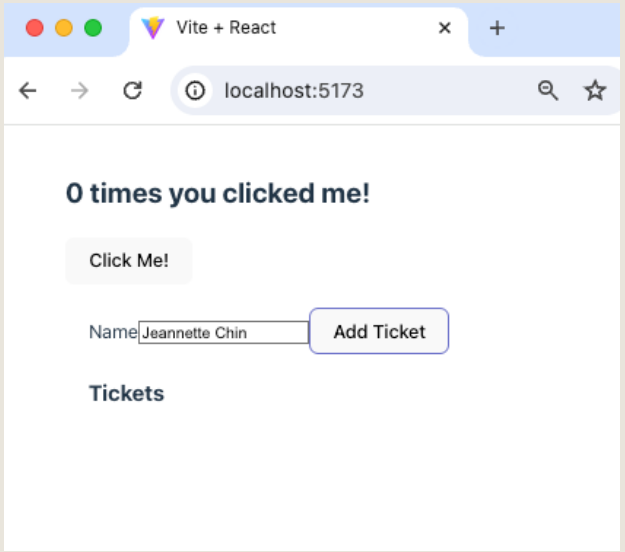
```jsx
  return (
    <div>
      <h2>{count} times you clicked me! </h2>
      <button onClick={handleClick}>Click Me!</button>
      {/* <div>
        <label>
          {" "}
          Name
          <input onChange={handleUser} value={user.name} />
        </label>
      </div> */}
      <div>
        <label>
          {" "}
          <span>Name</span>
          <input onChange={handleUser} value={user.name} />
        </label>
        <button onClick={addTicket}>Add Ticket</button>
      </div>
      <div>
        <h3>Tickets</h3>
        <ul>
          {tickets.map((ticket) => {
            return <li key={ticket.id}>{ticket.user.name}</li>;
          })}
        </ul>
      </div>
    </div>
  );
};

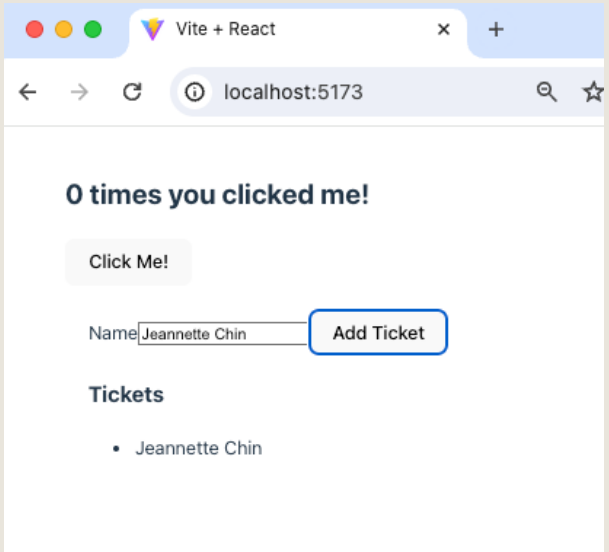
```

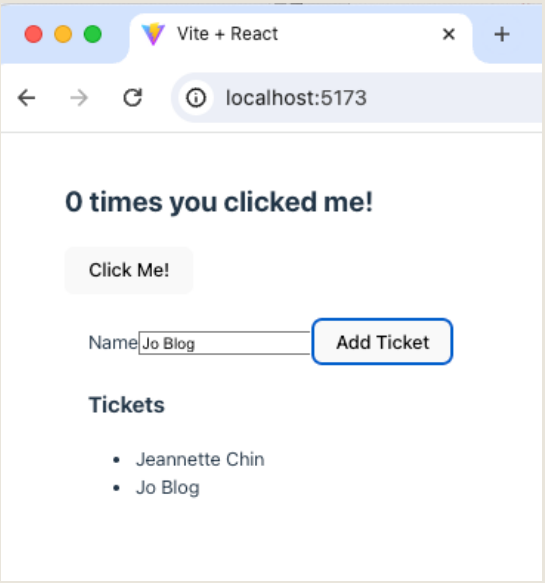
Updating Arrays in State – how?

- To render an array, use map()

```jsx
<h3>Tickets</h3>
<ul>
  {tickets.map((ticket) => {
    return <li key={ticket.id}>{ticket.user.name}</li>;
  })}
</ul>
```







Updating Arrays in State

	avoid (mutates the array)	prefer (returns a new array)
adding	`push`, `unshift`	`concat`, `[...arr]` spread syntax
removing	`pop`, `shift`, `splice`	`filter`, `slice`
replacing	`splice`, `arr[i] = ...` assignment	`map`
sorting	`reverse`, `sort`	copy the array first

Sharing state between components

- Sometimes, you want the state of two components to always change together.
- To do it, remove state from both of them, move it to their closest common parent, and then pass it down to them via props.
- This is known as “lifting state up”in REACT

Follow my code:

App.jsx | ShareState.jsx

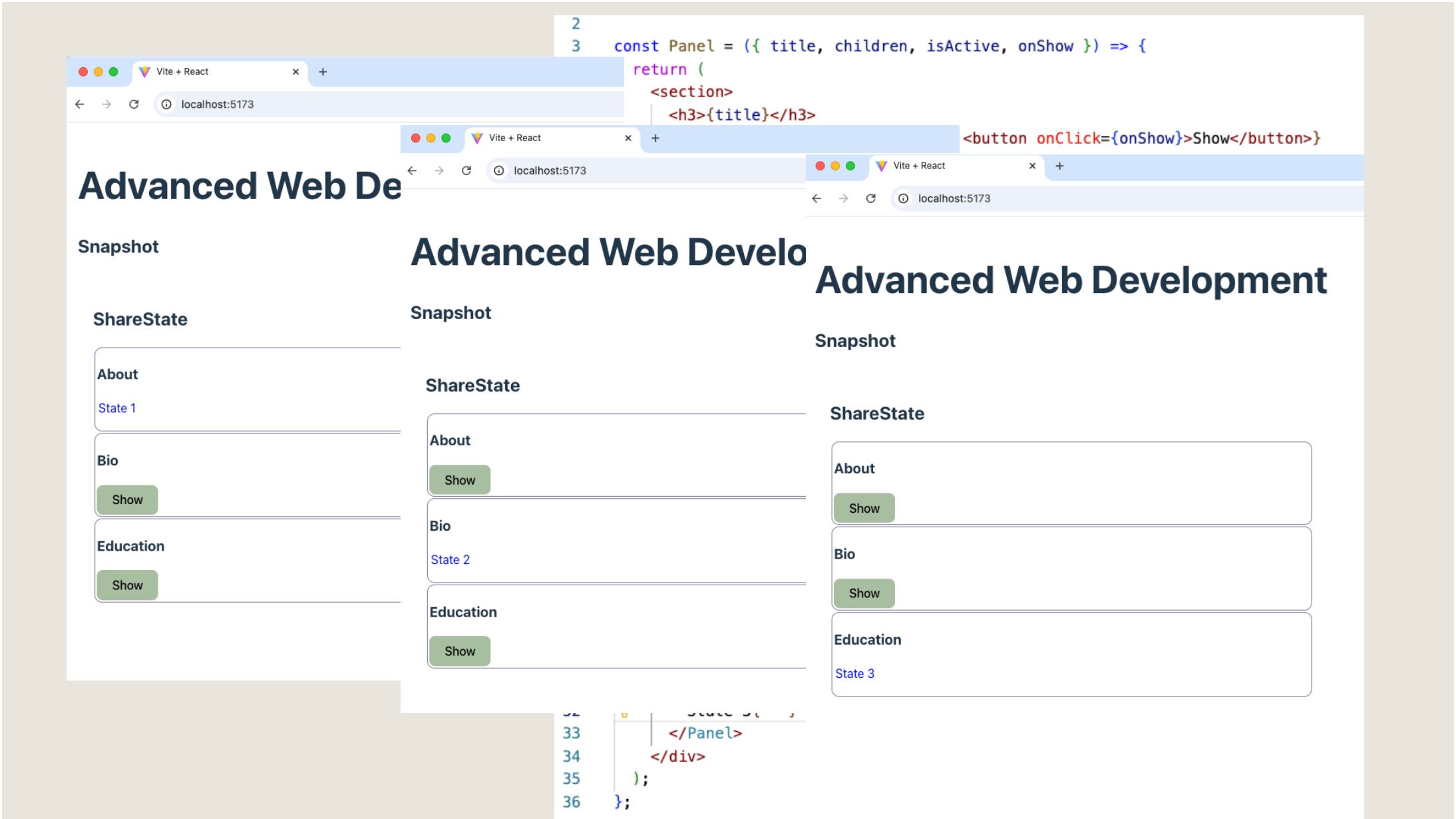
src > App.jsx > ...

```jsx
import Snapshot from "./Snapshot";
import Mutation from "./Mutation";
import ShareState from "./ShareState";

function App() {
  return (
    <>
      <h1>Advanced Web Development</h1>
      <h2>Snapshot</h2>
      {/* <Snapshot /> */}
      {/* <Mutation /> */}
      <ShareState />
    </>
  );
}

export default App;
```




Vite + React
localhost:5173
Advanced Web De
Snapshot
ShareState
About
State 1
Bio
Show
Education
Show
Vite + React
localhost:5173
Advanced Web Develo
Snapshot
ShareState
About
Show
Bio
State 2
Education
Show
Vite + React
localhost:5173
Advanced Web Development
Snapshot
ShareState
About
Show
Bio
Show
Education
State 3
2
3 const Panel = ({ title, children, isActive, onShow }) => {
return (
<section>
<h3>{title}</h3>
<button onClick={onShow}>Show</button>}
</Panel>

);
};

Sharing state between components

src > ShareState.tsx > ShareState

```tsx
import { useState, type ReactNode } from "react";

interface PanelProps {
  title: string;
  children: ReactNode;
  isActive: boolean;
  onShow: () => void;
}

const Panel = ({ title, children, isActive, onShow }: PanelProps) => {
  return (
    <section>
      <h3>{title}</h3>
      {isActive ? <p>{children}</p> : <button onClick={onShow}>Show</button>}
    </section>
  );
};
```

Sharing state between components

```jsx
/// parent
const ShareState = () => {
  const [active, setActive] = useState<number>(0);

  return (
    <div>
      <h2>ShareState</h2>
      <Panel title="About" isActive={active === 0} onShow={() => setActive(0)}>
        {" "}
        State 1{" "}
      </Panel>
      <Panel title="Bio" isActive={active === 1} onShow={() => setActive(1)}>
        {" "}
        State 2{" "}
      </Panel>
      <Panel
        title="Education"
        isActive={active === 2}
        onShow={() => setActive(2)}
      >
        {" "}
        State 3{" "}
      </Panel>
    </div>
  );
};
```

Practical Exercise and homework

- Lab2.1 – REACT TypeScript, Components, Props and JSX
- Lab2.2 – REACT TypeScript States

Next week

- Next.js & RTK